Leaf-Based Plant Disease Classification Using Machine Learning

UCSC398J

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Science

in

Computer Science

by

HARISH S

23BCS0151

Under the guidance of

Dr.JERART JULUS L

School of Computer Science Engineering and Information Systems

VIT, Vellore

April, 2026

DECLARATION

I hereby declare that the UCSC398J – Project thesis entitled "**Leaf-Based Plant Disease Classification Using Machine Learning**" submitted by me, for the award of the degree of ***Bachelor of Science in Computer Science, School of Computer Science Engineering and Information Systems*** to VIT is a record of bonafide work carried out by me under the supervision **Dr. JERART JULUS L, Assistant Professor Sr. Grade 1, SCORE, VIT, Vellore.**

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree ordiploma in this institute or any other institute or university.

Place: Vellore

Date: **Signature of the Candidate**

CERTIFICATE

This is to certify that the UCSC398J – Project thesis entitled "**Leaf-Based Plant Disease Classification Using Machine Learning**" submitted by **Harish S (23BCS0151**), **SCORE, VIT**, for the award of the degree of ***Bachelor of Science in Computer Science, School of Computer Science Engineering and Information Systems***, is a record of bonafide work carried out by him / her under my supervision during the period, 05. 12. 2025 to 17. 04. 2026, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The thesis fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: Vellore

Date:

Signature of the Guide

Dr.JERART JULUS L,

Assistant Professor Sr. Grade 1,

SCORE, VIT, Vellore.

Internal Examiner **External Examiner**

Head of the Department

Department of Computer Applications

ACKNOWLEDGEMENT

It is my pleasure to express with a deep sense of gratitude to my **UCSC398J - Project** guide **Dr. JERART JULUS L, Assistant Professor Sr. Grade 1,** School of Computer Science Engineering and Information Systems, Vellore Institute of Technology, Vellore for **his** constant guidance, continual encouragement, in my endeavor. My association with **him** is not confined to academics only, but it is a great opportunity on my part to work with an intellectual and an expert in the field of Artificial Intelligence and Educational Technology..

"I would like to express my heartfelt gratitude to Honorable Chancellor **Dr. G Viswanathan**; respected Vice Presidents **Mr. Sankar Viswanathan**, **Dr. Sekar Viswanathan**, Vice Chancellor **Dr. V. S. Kanchana Bhaaskaran**; Pro-Vice Chancellor **Dr. Partha Sharathi Mallick**, and Registrar **Dr. Jayabarathi T**.

My whole-hearted thanks to Dean Dr. Daphne Lopez, School of Computer Science Engineering and Information Systems; Head, Department of Smart Computing, Dr. Vijayan E; SCORE School Project Coordinator Dr. Jayaram Reddy; all faculty, staff and members working as limbs of our university for their continuous guidance throughout my course of study in unlimited ways.

It is indeed a pleasure to thank my parents and friends who persuaded and encouraged me to take up and complete my project successfully. Last, but not least, I express my gratitude and appreciation to all those who have helped me directly or indirectly towards the successful completion of the project.

Place: Vellore

Date: **Harish S**

Executive Summary

This project presents an intelligent plant disease classification system using deep learning techniques, specifically Convolutional Neural Networks (CNNs). The primary objective is to develop an automated solution that can accurately identify plant diseases from leaf images, thereby supporting early diagnosis and improving agricultural productivity.

The system utilizes a structured dataset of plant leaf images, divided into training, validation, and testing sets. Image preprocessing techniques such as resizing, normalization, and data augmentation are applied to enhance model performance and generalization. A custom CNN architecture is designed using TensorFlow/Keras, incorporating convolutional layers, batch normalization, dropout, and global average pooling for efficient feature extraction and classification.

The model is trained using the Adam optimizer and evaluated using metrics such as accuracy and top-3 accuracy. The system is capable of predicting diseases from user-uploaded images and provides the top predictions along with confidence scores. To enable real-world deployment, the trained model is converted into TensorFlow Lite (TFLite) format for use on Web and edge devices.

This project demonstrates the effectiveness of deep learning in agricultural applications, offering a fast, reliable, and scalable solution for plant disease detection.

CONTENTS PageNo.

Acknowledgement i

Executive Summary ii

Table of Contents iii

List of Figures ix

List of Tables xiv

Abbreviations xvi

Symbols and Notations xix

Chapter 1: Introduction 13

1.1 Objective 14
1.2 Motivation 14
1.3 Background 15

Chapter 2: Dissertation Description and Goals 16

2.1 Problem Statement 16
2.2 Scope of the Project 17
2.3 Gap Analysis 18
2.4 Proposed System 19

Chapter 3: Technical Specification **20**

3.1 Technology Stack 20
3.2 Functional Requirements 21
3.3 Non-Functional Requirements 21
3.4 Hardware Requirements 22

Chapter 4: Design Approach and Details **23**

4.1 Literature Survey 24
4.2 Findings from Literature Survey 25
4.3 System Architecture 26
4.3.1 Input / Image Acquisition Layer 27
4.3.2 Data Preprocessing Layer 27
4.3.3 CNN Model Processing Layer 27
4.3.4 Backend / API Layer 28

4.3.5 Data Storage Layer 28
4.4 UML Diagrams 29-31
4.5 System Modules 32
4.6 CNN Training Pipeline 32
4.7 Codes and Standards 33-35

Chapter 5: Schedule, Tasks and Milestones 36

5.1 Project Timeline 36
5.2 Phase-wise Description 37-39

Chapter 6: Dissertation Demonstration 40

6.1 Implementation Overview 40-42
6.2 Sample Code 42
6.2.1 CNN Model Architecture (Python / TensorFlow) 42-43
6.2.2 Image Preprocessing Pipeline (Python) 44
6.2.3 Flask REST API Backend (Python) 45-46
6.2.4 Disease Class List (Python — 75 classes) 46-47
6.2.5 Web Frontend—Image Upload & Prediction (HTML+JavaScript) 47-48
6.2.6 Dataset Loading & Augmentation Pipeline(Python) 49
6.2.7 Model Evaluation & Confusion Matrix (python) 50
6.2.8 Flask Route—Server Web Interface (Python) 51
6.2.9 Model Ecport for Deployment (Python) 52
6.3 System Screenshots 54-58

Chapter 7: Result and Discussion 59

7.1 Model Performance Evaluation 59
7.1.1 Dataset Overview 59
7.1.2 Model Architecture Summary 59
7.1.3 Training Configuration 60
7.2 Accuracy and Loss Analysis 61
7.2.1 Per-Class Performance 61
7.2.2 Overall Metrics Summary 61
7.2.3 Training and Validation Curves 62
7.2.4 Confidence Threshold Analysis 62
7.3 Cost Analysis 63
7.3.1 Development Cost 63
7.3.2 Hardware and Infrastructure Cost 63
7.3.3 Operational Cost Estimate 64
7.3.4 Comparative Cost vs Traditional Methods 64
7.4 Discussion 65
7.4.1 Summary of Achievements 65

7.4.2 Strengths 65
7.4.3 Limitations 65
7.4.4 Comparison with Existing Systems 66
7.4.5 Future Enhancements 66

Chapter 8: Summary 67

8.1 Summary of Findings 67
8.2 Recommendations 68
8.3 Conclusion 69

Chapter 9: References 70

Appendix A 71

List of Figures

Figure No.	Title	Page No.
Fig. 2.1	System Architecture of Plant Disease Detection System	8
Fig. 3.1	Data Preprocessing and Augmentation Pipeline	10
Fig. 3.2	CNN Model Architecture Diagram	11
Fig. 5.1	Model Training Accuracy and Loss Graph	14
Fig. 5.2	Model Evaluation Metrics Visualization	15
Fig. 6.1	Web Application Home Screen	18
Fig. 6.2	Image Upload and Prediction Interface	19
Fig. 6.3	Disease Prediction Output with Confidence Scores	20
Fig. 6.4	Top-5 Predictions Visualization	21
Fig. 6.5	System Workflow Diagram	22

List of Tables

Table No.	Title	Page No.
Table 2.1	Gap Analysis of Existing Systems vs Proposed System	6
Table 3.1	Technology Stack Used in the System	8
Table 3.2	Functional Requirements Specification	9
Table 3.3	Non-Functional Requirements	10
Table 4.1	Literature Survey of Plant Disease Detection Techniques	12
Table 5.1	Project Timeline and Milestones	15
Table 6.1	CNN Model Architecture Details	18
Table 7.1	Model Performance Metrics (Accuracy, Loss, Top-3 Accuracy)	21
Table 7.2	Comparison of Model Performance Before and After Optimization	22

List of Abbreviations

Abbreviation	Expanded Form
AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
CNN	Convolutional Neural Network
CV	Computer Vision
API	Application Programming Interface
UI	User Interface
GPU	Graphics Processing Unit
RGB	Red Green Blue
CNN	Convolutional Neural Network
ReLU	Rectified Linear Unit
BN	Batch Normalization
IoT	Internet of Things
TFLite	TensorFlow Lite
TF	TensorFlow
Keras	High-level Neural Network API
JSON	JavaScript Object Notation
HTTP	Hypertext Transfer Protocol
REST	Representational State Transfer
FPS	Frames Per Second

Symbols and Notations

Symbol / Notation	Description
x	Input image (leaf image)
y	Predicted output class (plant disease)
$f(x)$	CNN model function mapping input image to prediction
W	Weight matrix of the neural network
b	Bias term in neural network layers
$\hat{y}$	Predicted probability distribution
y_i	True label of the input image
$\hat{y}_i$	Predicted probability for class i
L	Loss function (cross-entropy loss)
η (eta)	Learning rate used during training
N	Total number of training samples
C	Number of classes (disease categories)
$H \times W \times D$	Image dimensions (Height × Width × Depth)
k	Kernel size in convolution layer
s	Stride value in convolution
p	Padding applied to input image
$A(x)$	Activation function (ReLU)
z	Linear combination ($z = Wx + b$)
$P(y$	$x)$
Dropout p_d	Dropout probability for regularization

CHAPTER 1

INTRODUCTION

Artificial Intelligence (AI) refers to the development of intelligent computer systems that can perform tasks typically requiring human intelligence. These tasks include problem-solving, decision-making, pattern recognition, and learning from experience. AI is a multidisciplinary field that encompasses machine learning, deep learning, computer vision, and natural language processing. Over the years, AI has evolved significantly, moving from theoretical concepts to practical applications that impact various domains such as healthcare, finance, transportation, and agriculture.

One of the most important subfields of AI is **machine learning**, which enables systems to learn from data and improve performance without being explicitly programmed. A further advancement of machine learning is **deep learning**, which uses artificial neural networks inspired by the human brain. Among these, **Convolutional Neural Networks (CNNs)** have proven to be highly effective for image processing and classification tasks.

In recent years, agriculture has emerged as a critical domain where AI can bring transformative changes. Agriculture plays a vital role in ensuring global food security and economic stability. However, plant diseases remain a major challenge affecting crop yield and quality. These diseases can spread rapidly and cause severe losses if not detected at an early stage. Traditional disease detection methods rely on manual inspection by agricultural experts, which is often time-consuming, expensive, and not feasible for large-scale farming.

With the advancement of computer vision and deep learning, it is now possible to automate plant disease detection using image-based analysis. Leaves of plants often exhibit visible symptoms such as discoloration, spots, and texture variations when affected by diseases. These visual patterns can be effectively analyzed using CNN models to classify diseases accurately.

This project focuses on developing an intelligent system for plant disease classification using Convolutional Neural Networks. The system is designed to process images of plant leaves and predict the type of disease present. The proposed solution integrates image preprocessing, deep learning-based classification, and deployment capabilities for real-world applications. By leveraging AI, this system aims to assist farmers and agricultural stakeholders in making faster and more accurate decisions.

Furthermore, the integration of web technologies enhances the accessibility of the system. Users can upload images of plant leaves through the web interface and receive instant predictions along with confidence scores. This not only reduces dependency on agricultural experts but also enables early detection, thereby preventing crop damage and improving productivity.

The project also emphasizes the importance of deploying AI models on edge devices. The trained CNN model is saved and deployed through a Flask-based web backend, enabling

efficient execution for real-time predictions. This ensures that the solution is lightweight, fast, and suitable for real-time applications in rural and remote areas.

Overall, this project demonstrates how artificial intelligence can be effectively applied in agriculture to address real-world challenges. It highlights the potential of deep learning in improving crop health monitoring and contributing to sustainable agricultural practices.

1.1Objective

The primary objective of this project is to develop an AI-powered plant disease detection system using deep learning techniques. The system aims to accurately classify plant diseases from leaf images and provide instant predictions to users.

The specific objectives of the project are as follows:

- To design and implement a Convolutional Neural Network (CNN) model for plant disease classification
- To preprocess and prepare a dataset of plant leaf images for training and testing
- To apply data augmentation techniques to improve model generalization
- To train and evaluate the model using appropriate performance metrics such as accuracy and top-3 accuracy
- To develop a user-friendly web interface for real-time disease prediction
- To display prediction results along with confidence scores
- To convert the trained model into TensorFlow Lite (TFLite) format for deployment on edge devices
- To provide a scalable and efficient solution for agricultural applications

1.2 Motivation

Agriculture is one of the most important sectors supporting global food production. However, plant diseases pose a significant threat to crop yield and quality. Early detection of diseases is crucial to minimize losses and ensure healthy crop growth. Unfortunately, traditional disease detection methods rely heavily on manual inspection by experts, which has several limitations.

Farmers often face challenges such as lack of access to agricultural experts, delayed diagnosis, and high costs associated with manual monitoring. In many rural areas, these challenges are even more pronounced, leading to reduced productivity and economic losses. Additionally, diseases can spread rapidly if not identified early, making timely intervention essential.

With the rapid advancements in artificial intelligence and deep learning, there is a strong motivation to develop automated systems that can assist in plant disease detection. CNN-based image classification models have shown remarkable performance in identifying

patterns and features in images. By leveraging these technologies, it is possible to build a system that can detect diseases accurately and efficiently.

Another key motivation for this project is the growing accessibility of the internet and web browsers. By deploying the model as a web application, users can easily upload images of plant leaves and receive instant predictions. This makes the system practical, scalable, and user-friendly.This makes the system practical, scalable, and user-friendly.

Furthermore, the Flask-based web deployment ensures that the system can operate efficiently on standard server hardware, making it accessible through any modern web browser.

This project is motivated by the vision of using AI to empower farmers, improve agricultural productivity, and contribute to sustainable farming practices. By providing a fast, reliable, and accessible solution, the system aims to bridge the gap between technology and agriculture.

1.3 Background

The concept of using artificial intelligence in agriculture has gained significant attention in recent years. Early systems focused on rule-based approaches and simple image processing techniques. However, these methods had limitations in handling complex patterns and variations in real-world data.

With the emergence of deep learning, particularly CNNs, the field of image classification has seen tremendous improvements. CNNs are specifically designed to process grid-like data such as images and are capable of automatically extracting relevant features. This makes them highly suitable for applications like plant disease detection.

Several studies have demonstrated the effectiveness of CNN-based models in classifying plant diseases with high accuracy. Public datasets such as PlantVillage have enabled researchers to train models on large collections of labeled images. These advancements have paved the way for developing practical solutions that can be deployed in real-world scenarios.

In addition to model development, there has been a growing focus on deploying AI systems through web-based platforms. Technologies like Flask allow trained models to be served via web APIs, making them accessible through standard browsers for on-field applications.

This project builds upon these advancements by integrating deep learning, Web technology, and real-time prediction capabilities into a single system. It aims to provide a comprehensive solution for plant disease detection that is both efficient and accessible.

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

The Plant Disease Detection system is an AI-powered application designed to identify and classify plant diseases using deep learning techniques, specifically Convolutional Neural Networks (CNNs). The system analyzes images of plant leaves to detect visible symptoms such as discoloration, spots, and texture variations, which are key indicators of diseases. By leveraging computer vision and machine learning, the project aims to automate the process of plant disease diagnosis and provide accurate results in real time.

The system is developed using a structured pipeline that includes image acquisition, preprocessing, model training, and prediction. A labeled dataset of plant leaf images is used to train the CNN model, enabling it to learn complex patterns associated with different diseases. The trained model is then integrated into a user-friendly interface where users can upload or capture images and receive instant predictions along with confidence scores The deployment of the model through a Flask web backend ensures that the system can run efficiently and be accessed through any modern web browser, making it accessible even in resource-constrained environments.

The primary goal of this project is to develop an accurate, reliable, and scalable solution for plant disease detection. It aims to assist farmers and agricultural stakeholders in identifying diseases at an early stage, thereby reducing crop losses and improving productivity. Additionally, the project seeks to minimize dependency on manual inspection and expert intervention by providing an automated and easy-to-use system. Overall, this project demonstrates the practical application of artificial intelligence in agriculture and contributes to the advancement of precision farming technologies.

2.1 Problem Statement

Agriculture is a fundamental sector that supports global food production and economic stability. However, plant diseases remain one of the major challenges affecting crop yield and quality. Diseases caused by fungi, bacteria, viruses, and environmental factors can spread rapidly and cause severe damage if not detected at an early stage.

Traditional plant disease detection methods rely heavily on manual inspection by agricultural experts. Farmers visually examine crops to identify symptoms such as discoloration, spots, or deformities in leaves. While this approach can be effective in some cases, it has several limitations:

- It is time-consuming and labor-intensive
- Requires expert knowledge and experience
- Not scalable for large agricultural fields
- Prone to human error and misdiagnosis
- Limited accessibility in rural and remote areas

In many regions, farmers do not have immediate access to agricultural specialists. As a result, diseases often go undetected until they reach an advanced stage, leading to significant crop losses. Additionally, environmental variations and similar visual symptoms across different diseases make accurate identification difficult.

With the increasing demand for food production and sustainable farming practices, there is a strong need for an automated, accurate, and accessible system for plant disease detection. Such a system should be capable of analyzing plant images, identifying diseases, and providing quick results to assist farmers in taking timely action.

This project addresses these challenges by developing a deep learning-based plant disease classification system that uses Convolutional Neural Networks (CNNs) to automatically detect diseases from leaf images.

2.2 Scope of the Project

The scope of this project focuses on designing and implementing an AI-based system for plant disease classification using image processing and deep learning techniques. The system is intended to provide an efficient and user-friendly solution for identifying plant diseases.

Key Functional Scope

The system is capable of:

- Detecting plant diseases from leaf images
- Classifying multiple disease categories across different crops
- Providing predictions with confidence scores
- Supporting real-time image-based disease detection
- Displaying top predicted classes for better understanding

Technical Scope

- Use of Convolutional Neural Networks (CNNs) for image classification
- Implementation using TensorFlow/Keras framework
- Image preprocessing including resizing, normalization, and augmentation
- Model training, validation, and testing using labeled datasets
- Deployment using Flask web framework for browser-based access

Application Scope

- Browser-based web application interface
- Real-time prediction using uploaded or captured images
- Accessible to farmers, students, and researchers

Limitations of Scope

- The system focuses only on leaf-based disease detection
- Performance depends on image quality and dataset diversity
- Limited to diseases included in the training dataset
- Does not provide detailed treatment recommendations

Future Scope

- Integration with IoT-based smart agriculture systems
- Real-time camera-based disease detection
- Expansion to more crops and diseases
- Explainable AI (XAI) for better interpretation of predictions

2.3 Gap Analysis

Despite significant advancements in plant disease detection using machine learning and deep learning, several limitations still exist in current systems. This section identifies the gaps in existing approaches and highlights how the proposed system addresses them.

S.No	Gap Identified	Existing System Limitation	Proposed System Solution
1	Manual Detection	Time-consuming and requires experts	Automated AI-based detection
2	Limited Accessibility	Farmers lack expert support in rural areas	Web-based application
3	Dataset Limitations	Models trained on controlled datasets	Use of diverse datasets
4	Low Accuracy	Traditional ML models less effective	CNN-based deep learning model
5	Lack of Real-Time Detection	Offline or delayed processing	Instant prediction system
6	No Confidence Scores	Limited interpretability	Displays prediction probabilities
7	Single Crop Focus	Limited applicability	Multi-crop disease classification
8	Deployment Issues	High computational requirements	Flask-based lightweight web deployment

Analysis of Gaps

Most traditional systems rely on handcrafted features and shallow machine learning models, which are less effective in handling complex image patterns. Additionally, many research-based systems are not deployed as practical applications, limiting their usability in real-world scenarios.

Another major gap is the lack of Web-friendly solutions. Farmers need tools that are easy to use and accessible in field conditions. The absence of real-time prediction capabilities further reduces the effectiveness of existing systems.

The proposed system addresses these gaps by integrating deep learning, Web deployment, and real-time prediction capabilities into a single platform.

2.4 Proposed System

The proposed system is an AI-based plant disease detection platform that utilizes Convolutional Neural Networks (CNNs) to automatically identify and classify plant diseases from leaf images. The system is designed to provide a fast, accurate, and user-friendly solution for diagnosing plant health conditions, thereby supporting farmers and agricultural stakeholders in making timely decisions.

The system follows a structured workflow beginning with image acquisition, where users upload images of plant leaves through a web browser interface. The input images are then passed to the preprocessing stage, where they are resized, normalized, and enhanced to ensure consistency and improve model performance. Data augmentation techniques such as rotation, flipping, and zooming are also applied during training to increase dataset diversity and reduce overfitting.

Once preprocessed, the images are fed into a trained CNN model that performs feature extraction and classification. The convolutional layers identify important visual patterns such as color variations, spots, and textures associated with different plant diseases. These features are further processed through pooling and fully connected layers, and the final output is generated using a softmax activation function, which provides probability scores for each disease class.

The system then displays the predicted disease along with confidence scores and the top possible predictions. For practical deployment, the trained model is served through a Flask web API, enabling efficient browser-based access for real-time predictions.

Overall, the proposed system offers an automated, scalable, and real-time solution for plant disease detection, reducing dependency on manual inspection and improving agricultural productivity through early and accurate diagnosis.

CHAPTER 3

TECHNICAL SPECIFICATION

This chapter describes the technical requirements and specifications of the Plant Disease Detection system. It includes the technology stack, functional and non-functional requirements, and hardware requirements necessary for the development and deployment of the system.

3.1 Technology Stack

The system is developed using a combination of modern tools and technologies that support deep learning, image processing, and Web/web deployment. The technology stack ensures efficiency, scalability, and real-time performance.

The core of the system is built using Python along with TensorFlow and Keras for deep learning model development. Image processing is handled using OpenCV and NumPy, while Flask is used for backend API services. The frontend is developed using HTML, CSS, and JavaScript to provide a browser-based web interface. The trained model is loaded and served directly through the Flask backend to enable efficient real-time predictions via the web.

Table 3.1 Technology Stack

Component	**Technology**	**Purpose**
Programming Language	Python	Model development and backend processing
ML Framework	TensorFlow / Keras	Building and training CNN model
Image Processing	OpenCV, NumPy, PIL	Image preprocessing and manipulation
Backend	Flask	API development and model integration
Frontend	HTML, CSS, JavaScript	Web application interface
Deployment	Flask+ SavedModel/H5	Web server deployment
Development Tools	Google Colab, VS Code	Model training and coding
Version Control	Git / GitHub	Source code management

3.2 Functional Requirements

Functional requirements define the core functionalities of the system. These requirements describe what the system should do to meet user needs.

The Plant Disease Detection system allows users to upload or capture images of plant leaves and receive disease predictions. The system processes the image, classifies the disease using a trained CNN model, and displays the results along with confidence scores.

Table 3.2 Functional Requirements

ID	Requirement	Description	Priority
FR1	Image Upload	Users can upload leaf images through the web browser	High
FR2	Image Capture	Users can capture images using camera	High
FR3	Image Preprocessing	System resizes and normalizes images	High
FR4	Disease Detection	CNN model classifies plant diseases	High
FR5	Prediction Output	Displays disease name and confidence score	High
FR6	Top Predictions	Shows top-5 predicted classes	Medium
FR7	User Interface	Provides simple and user-friendly UI	Medium
FR8	Model Deployment	Supports browser-based inference via Flask API	Medium

3.3 Non-Functional Requirements

Non-functional requirements define the quality attributes of the system such as performance, usability, reliability, and scalability.

The system is designed to provide fast and accurate predictions while maintaining ease of use and accessibility for users. It should perform efficiently even on Web devices with limited resources.

Table 3.3 Non-Functional Requirements

Attribute	Requirement	Measurement Criteria
Performance	Fast prediction time	Response time < 2 seconds
Accuracy	High classification accuracy	Accuracy > 85%
Usability	Easy-to-use interface	Simple UI, minimal user steps
Reliability	Consistent results	Stable predictions across inputs
Scalability	Support multiple users	Handles concurrent requests
Portability	Web compatibility	Runs on any modern browser
Availability	System accessibility	Available anytime (24/7)

3.4 Hardware Requirements

The Plant Disease Detection system is designed to run on standard hardware devices. It does not require high-end infrastructure for end users, making it suitable for practical deployment in real-world agricultural environments.

For End Users:

- Smartphone or computer with camera
- Internet connection (optional for cloud-based processing)
- Modern web browser or Web application

For Development and Training:

- Processor: Intel Core i5 / i7 or equivalent
- RAM: Minimum 8 GB (16 GB recommended)
- GPU: NVIDIA GPU (or Google Colab GPU for training)
- Storage: At least 10–20 GB free space

The use of TensorFlow Lite ensures that the trained model can run efficiently on Web devices with limited computational resources. This makes the system highly accessible and suitable for deployment in rural and remote areas.

CHAPTER 4

DESIGN APPROACH AND DETAILS

4.1 Literature Survey

A comprehensive literature survey was conducted to analyze existing research and advancements in plant disease detection using machine learning and deep learning techniques. The primary objective of this review is to understand the strengths, limitations, and trends in current approaches, and to identify research gaps that can be addressed in the proposed system.

Plant disease detection has evolved significantly over the years, moving from traditional manual inspection methods to automated systems powered by artificial intelligence. Early approaches relied on image processing techniques combined with classical machine learning algorithms such as Support Vector Machines (SVM), Random Forest, and k-Nearest Neighbors (k-NN). These methods required manual feature extraction, which limited their ability to handle complex patterns in plant images.

With the advancement of deep learning, particularly Convolutional Neural Networks (CNNs), plant disease detection systems have achieved significant improvements in accuracy and efficiency. CNN-based models automatically learn hierarchical features from images, making them highly suitable for identifying disease patterns such as leaf discoloration, spots, and texture variations. Studies such as those by Mohanty et al. (2016) demonstrated the effectiveness of deep learning in multi-class plant disease classification with high accuracy.

Recent research has focused on improving model performance through advanced architectures such as DenseNet and EfficientNet. These models provide better accuracy while reducing the number of parameters, making them suitable for real-world applications. Additionally, the integration of explainable AI (XAI) techniques has enhanced the interpretability of deep learning models, allowing users to understand the reasoning behind predictions.

Despite these advancements, several challenges remain. Many existing systems are trained on datasets collected in controlled environments, which may not generalize well to real-world conditions. Variations in lighting, background, and image quality can significantly affect model performance. Furthermore, most research focuses on model development without providing practical deployment solutions such as Web applications.

Another important observation is the growing trend of Web-based plant disease detection systems. These systems enable farmers to capture leaf images and receive instant predictions, improving accessibility and usability. However, real-time deployment on resource-constrained devices requires model optimization techniques such as TensorFlow Lite (TFLite).

Overall, the literature survey highlights that deep learning, particularly CNN-based approaches, is the most effective method for plant disease detection. However, there is a need for systems that are not only accurate but also practical, scalable, and accessible. The proposed project aims to address these challenges by developing a CNN-based plant disease detection system integrated with a web application for real-time browser-based use.

Table 4.1 Literature Survey

S.No	Title	Author & Year	Merits	Demerits
1	Plant Disease Detection Using Deep Learning	D. Tejaswi et al., 2024	Uses CNN for accurate detection; supports multiple crops; real-time Web solution	Depends on image quality; limited real-world testing
2	Predicting Diabetes Mellitus With Machine Learning Techniques	Quan Zou et al., 2018	Provides large dataset useful for ML training	Data collected in controlled conditions; poor generalization
3	Plant Disease Detection and Classification Techniques: A Comparative Study	Wubetu Demilie, 2024	Compares ML and DL techniques; identifies best-performing models	No real-time implementation; lacks practical system
4	Systematic Analysis of ML and DL for Plant Disease Detection	Various Authors	Highlights effectiveness of CNN using PlantVillage dataset	Focuses on analysis only; depends on dataset quality
5	Plant Disease Detection Using Machine Learning	Shima Ramesh et al., 2018	Uses Random Forest and texture features; simple approach	Lower accuracy than deep learning; sensitive to noise
6	Using Deep Learning for Image-Based Plant Disease Detection	Mohanty et al., 2016	High accuracy using CNN; multi-class classification	Limited real-world validation
7	Fine-Tuning Deep Learning Models for Plant Disease Identification	Too et al., 2019	DenseNet improves accuracy with fewer parameters	Requires high computational resources
8	Research on Plant Disease Identification Based on CNN	Xuewei Sun et al., 2022	EfficientNet with attention improves performance	Complex model design
9	Diabetes Prediction Using Ensemble ML Models	Hasan et al., 2020	Shows benefits of ensemble learning	Not specific to plant disease domain
10	Improving Plant	Natasha	High accuracy with	Requires large

S.No	Title	Author & Year	Merits	Demerits
	Disease Classification with XAI	Nigar et al., 2024	explainable AI; improves trust	datasets; high computation cost.

4.2 Findings from Literature Survey

Based on the detailed review of existing research in plant disease detection using machine learning and deep learning techniques, several important observations and insights have been identified. These findings help in understanding the current state of technology and highlight the need for improved solutions.

One of the key findings is that **deep learning models, particularly Convolutional Neural Networks (CNNs), outperform traditional machine learning techniques**. Earlier approaches using algorithms such as Random Forest and Support Vector Machines required manual feature extraction, which limited their ability to capture complex patterns in plant leaf images. In contrast, CNNs automatically learn hierarchical features, resulting in higher accuracy and better generalization.

Another significant observation is that **dataset quality plays a crucial role in model performance**. Many existing studies use datasets collected under controlled conditions, such as the PlantVillage dataset. While these datasets help achieve high accuracy during testing, models trained on such data often struggle to perform well in real-world environments where lighting, background, and image quality vary.

The literature also reveals that **CNN architectures such as DenseNet, EfficientNet, and other deep models have improved classification accuracy**. However, these models often require high computational resources, making them difficult to deploy on Web or edge devices. This creates a gap between research-level models and real-world applications.

It is also observed that **most existing systems focus primarily on model development and lack practical deployment solutions**. Very few systems provide Web or web-based interfaces that allow users to interact with the model in real time. This limits the usability of such systems for farmers and end users.

Another important finding is the **increasing trend toward Web-based plant disease detection systems**, which improve accessibility for farmers. However, deploying models on Web devices requires optimization techniques such as TensorFlow Lite (TFLite) to ensure efficient performance.

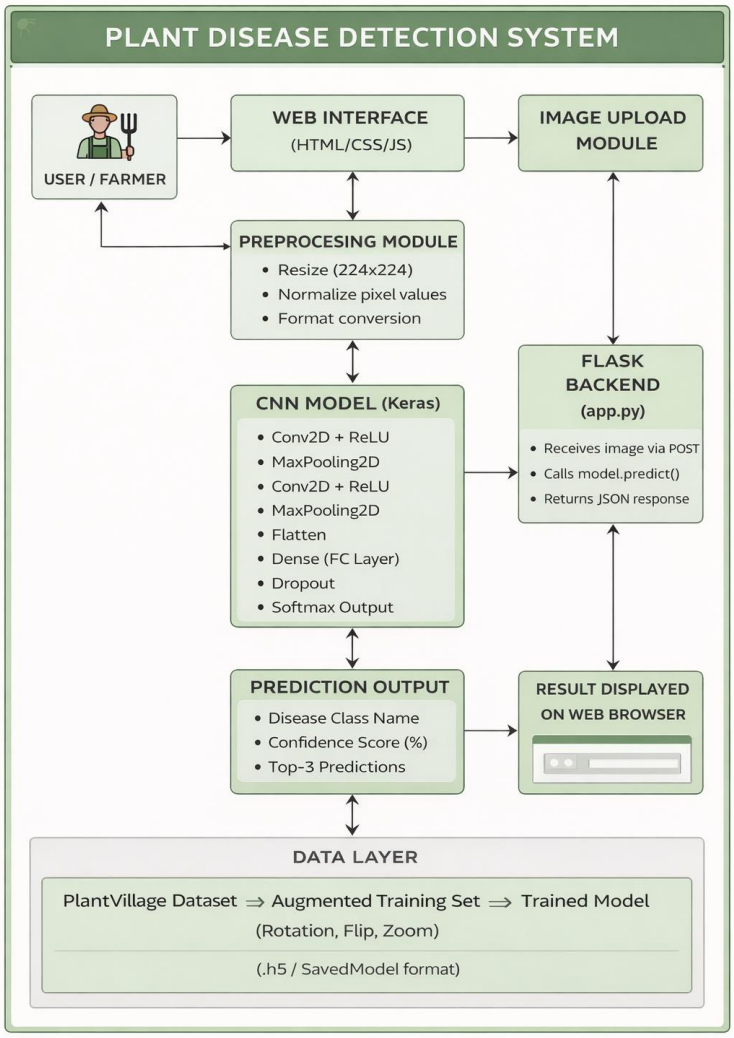
Additionally, many systems **lack robustness in handling real-world inputs**, such as images with complex backgrounds or non-leaf objects. This highlights the need for more generalized and reliable models.

4.3 System Architecture

The Plant Disease Detection system is designed using a modular and layered architecture that ensures scalability, efficiency, and ease of deployment. The system integrates image processing, deep learning, and Web/web technologies to provide real-time disease prediction.

The architecture consists of multiple layers, each responsible for a specific function in the system workflow. The overall process begins with image acquisition and ends with disease prediction output. The system is designed to handle user inputs efficiently and provide accurate results using a trained Convolutional Neural Network (CNN) model.

The architecture ensures separation of concerns, allowing individual components such as preprocessing, model inference, and user interaction to operate independently. This modular design improves maintainability and scalability of the system.



4.3.1 Input / Image Acquisition Layer

The Input Layer is responsible for capturing or receiving plant leaf images from the user. This is the first stage of the system where raw data is collected.

Users can provide input in two ways:

- Uploading images from the device gallery

The input images may vary in size, resolution, lighting conditions, and background complexity. Therefore, this layer must handle diverse image formats and ensure that the images are correctly passed to the preprocessing stage.

This layer plays a crucial role in ensuring that the system is user-friendly and accessible. A well-designed interface allows users, especially farmers, to easily capture and submit images for analysis.

4.3.2 Data Preprocessing Layer

The Data Preprocessing Layer is responsible for preparing the input images before feeding them into the CNN model. Proper preprocessing is essential for improving model accuracy and consistency.

The preprocessing steps include:

- **Image Resizing:** All images are resized to a fixed dimension (e.g., 224 × 224 pixels)
- **Normalization:** Pixel values are scaled to a range of 0 to 1
- **Noise Reduction:** Unwanted distortions in images are minimized
- **Data Augmentation (during training):**
    - Rotation
    - Flipping
    - Zooming
    - Contrast adjustment

These steps ensure that the model receives standardized input, which improves learning and generalization. Data augmentation also helps in reducing overfitting and increases the robustness of the model.

4.3.3 CNN Model Processing Layer

This layer is the core component of the system, where the actual disease classification takes place. A Convolutional Neural Network (CNN) is used to extract features from the input images and classify them into different disease categories.

The CNN model consists of multiple layers, including:

- **Convolutional Layers:** Extract features such as edges, textures, and patterns
- **Batch Normalization:** Stabilizes and accelerates training
- **Activation Function (ReLU):** Introduces non-linearity
- **Pooling Layers:** Reduce spatial dimensions and computation
- **Dropout Layers:** Prevent overfitting
- **Fully Connected Layers:** Perform classification

The final output layer uses a **Softmax function** to generate probability scores for each disease class. The model then predicts the disease with the highest probability.

This layer is responsible for achieving high accuracy in disease detection and is trained using a labeled dataset of plant leaf images.

4.3.4 Backend / API Layer

The Backend Layer acts as a bridge between the frontend interface and the CNN model. It is responsible for handling communication, processing requests, and returning predictions.

The backend is implemented using **Flask**, a lightweight Python web framework. Its responsibilities include:

- Receiving image data from the frontend
- Sending the image to the preprocessing module
- Passing processed data to the CNN model
- Generating predictions
- Enabling integration with web browsers and web applications

The backend ensures smooth and secure communication between different components of the system. It also enables integration with Web and web applications.

4.3.5 Data Storage Layer

The Data Storage Layer manages the storage of datasets, model files, and prediction results.

This layer includes:

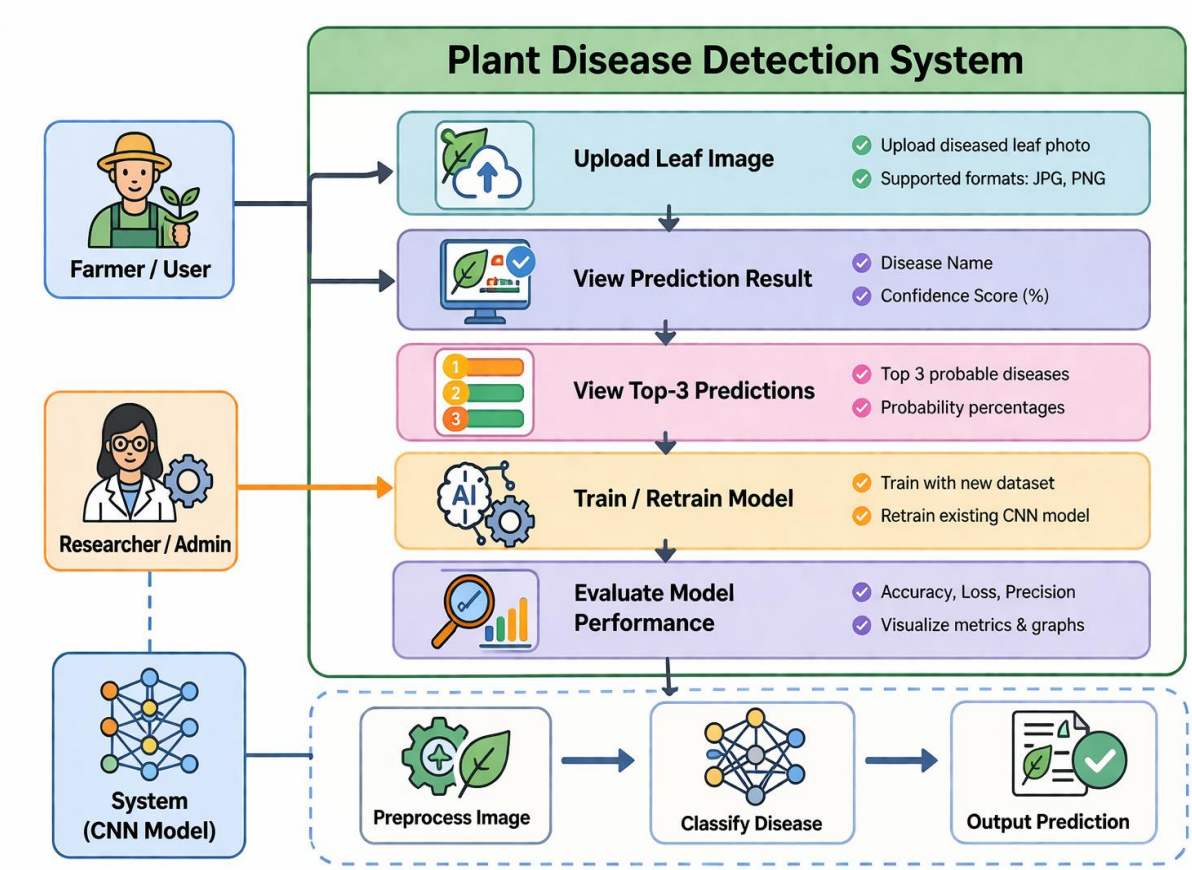
- **Training Dataset:** Collection of labeled plant leaf images
- **Model Storage:** Saved trained CNN model (.h5 or . SavedModel format)
- **Prediction Logs (optional):** Stores past predictions for analysis

Efficient data management is important for training the model and maintaining system performance. The trained model is stored and loaded during inference to provide real-time predictions.

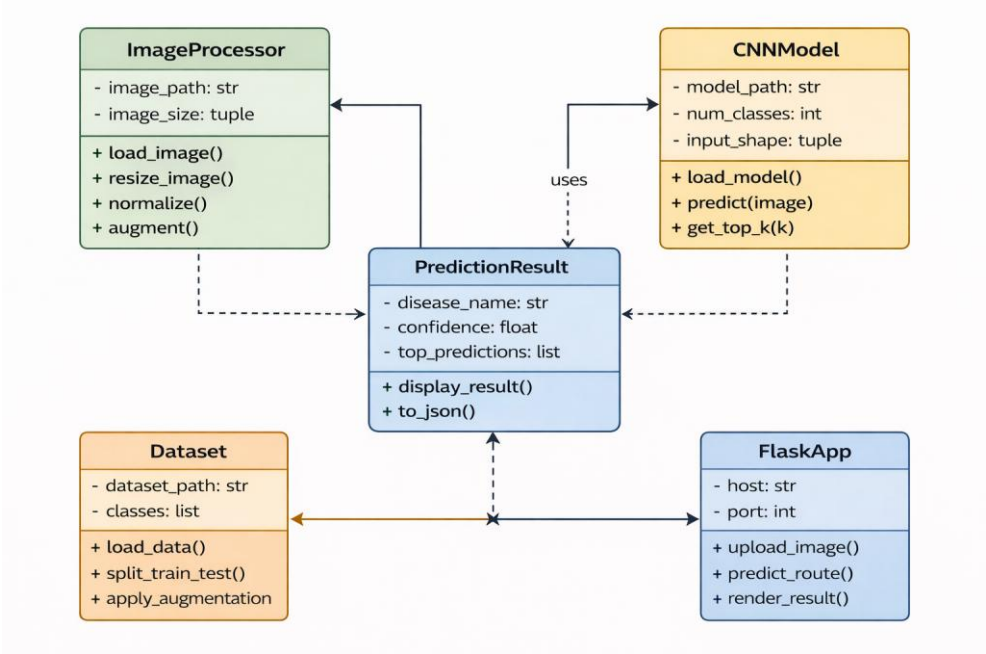
4.4 UML Diagrams

Unified Modeling Language (UML) diagrams are used to visually represent the structure and behavior of a system. They help in understanding the interaction between different components and provide a clear overview of system functionality. In this project, UML diagrams are used to model the plant disease detection system and illustrate how users interact with the system.

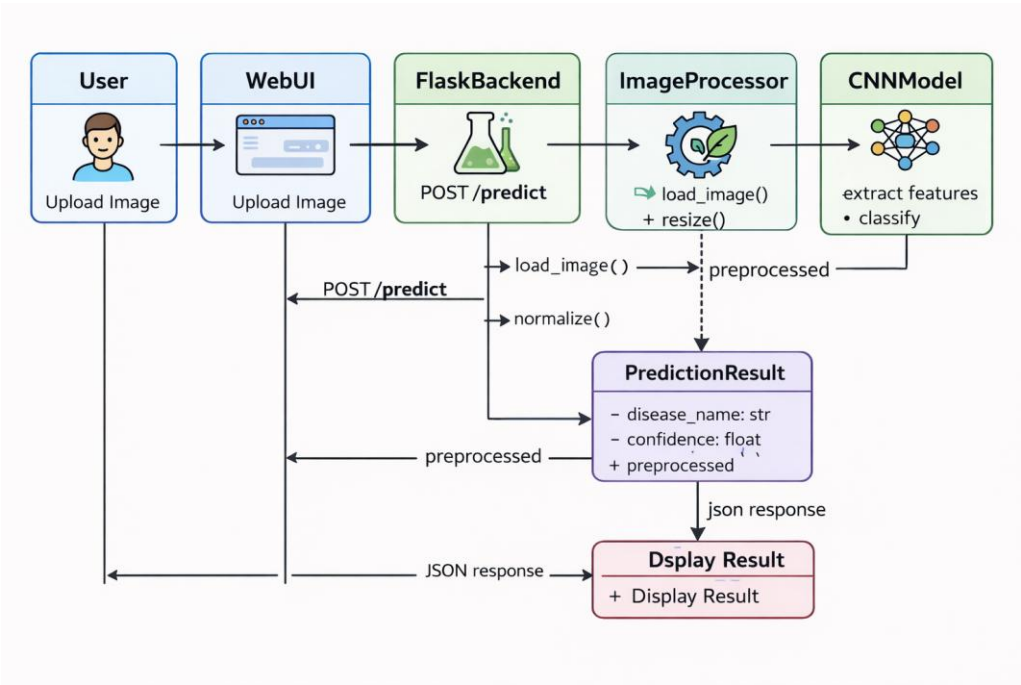
1. Use Case Diagram



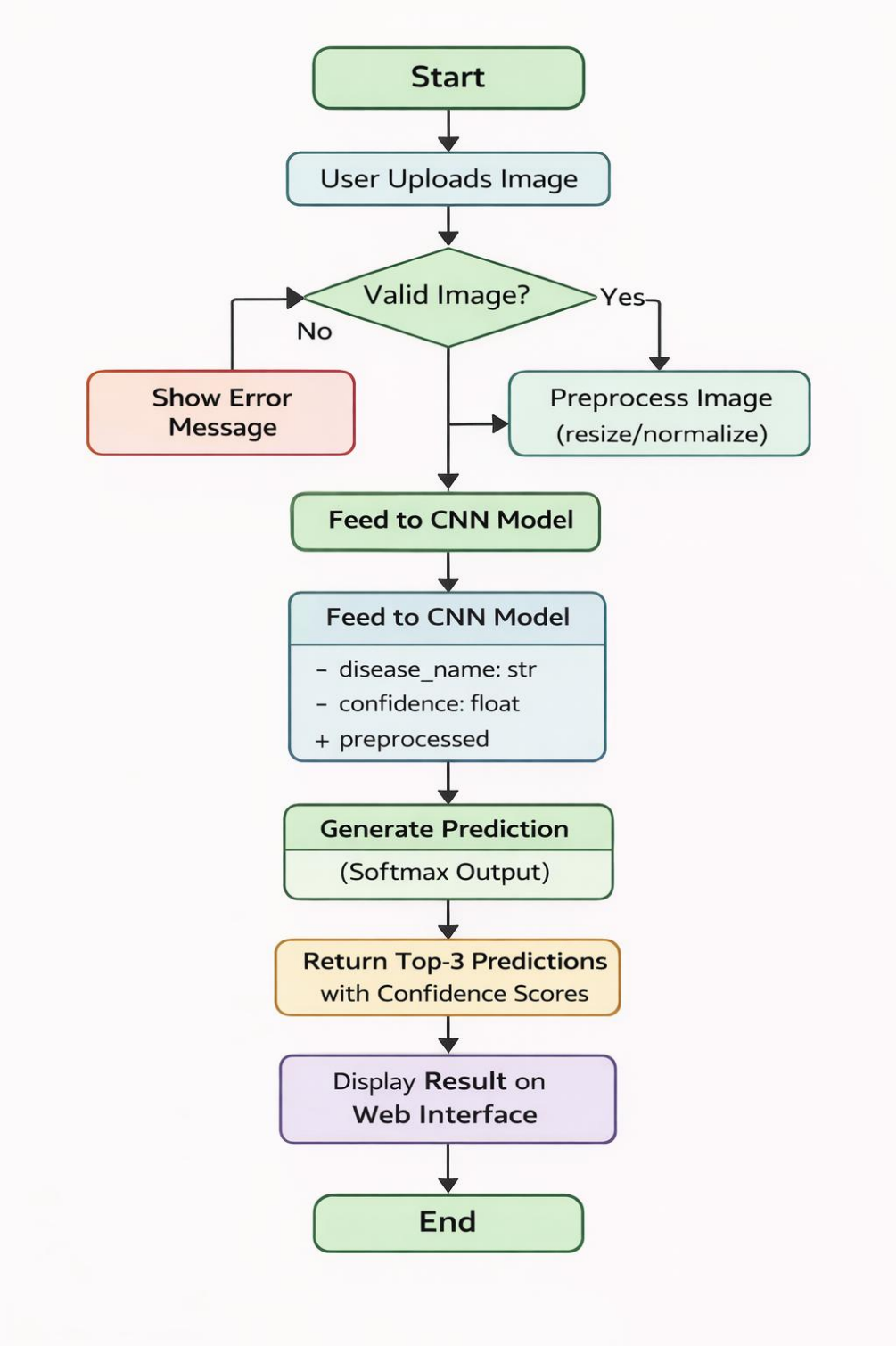
2. Class Diagram



3. Sequence Diagram



4. Activity Diagram



4.5 System Modules

The Plant Disease Detection system is designed using a modular approach, where the entire system is divided into smaller functional units called modules. Each module is responsible for a specific task, ensuring better organization, scalability, and maintainability of the system.

The modular design allows independent development and testing of each component, making the system more efficient and flexible. The major modules of the system are described below.

Table 4.2 System Modules and Features

S.No	Module Name	Description	Key Functions
1	Image Acquisition	Collects input images from users	Capture/upload leaf images
2	Data Preprocessing	Prepares images for model input	Resize, normalize, augment
3	CNN Model	Core model for classification	Feature extraction and prediction
4	Prediction Module	Generates output results	Disease prediction, confidence score
5	Backend / API	Handles communication	Request processing, model integration
6	User Interface	Provides user interaction	Display results, upload images
7	Model Deployment	Enables Web deployment	Flask serving, model optimization

4.6 CNN Training Pipeline

The CNN training pipeline is a crucial component of the plant disease detection system, responsible for building a robust and accurate deep learning model. It defines the sequence of steps involved in preparing data, training the model, and evaluating its performance.

The pipeline begins with **dataset collection and organization**, where plant leaf images are gathered from publicly available datasets and real-world sources. The dataset is structured into training, validation, and testing sets to ensure proper model learning and evaluation.

The next step is **data preprocessing**, which includes resizing images to a fixed dimension (e.g., 224 × 224 pixels), normalizing pixel values to a range of 0 to 1, and removing noise. To improve model generalization, **data augmentation techniques** such as rotation, flipping,

zooming, and contrast adjustment are applied. These techniques increase dataset diversity and reduce overfitting.

Once preprocessing is completed, the data is fed into the **CNN model training phase**. The model consists of multiple convolutional layers for feature extraction, followed by pooling layers for dimensionality reduction. Batch normalization and dropout layers are used to improve training stability and prevent overfitting. The extracted features are passed through fully connected layers, and the final classification is performed using a softmax activation function.

The model is compiled using the **Adam optimizer** and trained with the **sparse categorical cross-entropy loss function**. During training, performance metrics such as accuracy and top-3 accuracy are monitored. The validation dataset is used to evaluate the model after each epoch, helping to detect overfitting and optimize performance.

After training, the model undergoes **evaluation using the test dataset**, where metrics such as accuracy and loss are calculated. This step ensures that the model performs well on unseen data.

Finally, the trained model is saved in H5 or SavedModel format and deployed through the Flask backend for real-time web-based predictions. This enables efficient real-time predictions with reduced computational requirements.

Overall, the CNN training pipeline ensures that the model is accurate, reliable, and suitable for practical applications in plant disease detection.

4.7 Codes and Standards

Codes and standards play a vital role in ensuring the quality, reliability, maintainability, and scalability of software systems. In the Plant Disease Detection system, standard coding practices and development guidelines are followed to ensure that the system is efficient, secure, and easy to maintain.

The project follows industry-standard practices in software development, machine learning implementation, and system design. These standards help in improving code readability, reducing errors, and ensuring consistency across different components of the system.

1. Coding Standards

The system is primarily developed using Python, and standard Python coding conventions are followed based on **PEP 8 guidelines**. These include:

- Proper naming conventions for variables, functions, and classes
- Use of meaningful and descriptive identifiers
- Consistent indentation and formatting

- Modular programming with reusable functions
- Proper commenting and documentation

Following these practices ensures that the code is easy to understand, debug, and maintain.

2. Machine Learning Standards

For implementing the CNN model, best practices in deep learning are followed:

- Use of standardized frameworks such as TensorFlow and Keras
- Proper dataset splitting into training, validation, and testing sets
- Use of data augmentation techniques to improve generalization
- Monitoring of training using metrics such as accuracy and loss
- Avoiding overfitting through dropout and regularization

These standards ensure that the model is reliable and performs well on unseen data.

3. Data Handling Standards

Proper data handling practices are followed to maintain data quality and consistency:

- Images are resized to a uniform dimension (e.g., 224 × 224 pixels)
- Pixel values are normalized for better model performance
- Dataset is organized into structured directories
- Data augmentation is applied to increase diversity

These practices help in improving model accuracy and robustness.

4. API and Communication Standards

The backend of the system is implemented using Flask, and standard RESTful API practices are followed:

- Use of HTTP methods such as GET and POST
- Data exchange in JSON format
- Clear request and response structures
- Proper error handling and validation

These standards ensure smooth communication between the frontend and backend.

5. User Interface Standards

The frontend is developed using HTML, CSS, and JavaScript, following UI/UX design principles:

- Simple and intuitive user interface
- Minimal user input steps
- Clear display of prediction results
- Responsive design for different screen sizes

This ensures that the system is user-friendly and accessible to non-technical users.

6. Security Standards

Basic security practices are implemented to ensure safe usage of the system:

- Input validation to prevent invalid data
- Secure handling of user inputs
- Protection against common vulnerabilities

Although the system is primarily for academic purposes, these practices improve system reliability.

7. Deployment Standards

For deployment, the model is served through the Flask web framework, following standard optimization practices:

- Efficient model loading and inference
- Clean RESTful API design
- Compatibility with standard web browsers

This ensures that the system performs efficiently in real-world scenarios.

CHAPTER 5

SCHEDULE, TASKS AND MILESTONES

5.1 Project Timeline

The project timeline defines the sequence of activities carried out during the development of the Plant Disease Detection system. It provides a structured plan that ensures systematic execution of each phase, from data collection to final deployment. The timeline helps in tracking progress, managing tasks efficiently, and meeting project deadlines.

The project is divided into multiple phases, each focusing on a specific aspect of system development. These phases include dataset preparation, preprocessing, model development, evaluation, application development, and integration.

Project Phases Overview

The development of the system is carried out in the following phases:

1. Dataset Collection
2. Data Preprocessing
3. Model Development
4. Model Training and Evaluation
5. Application Development
6. Integration and Testing
7. Final Deployment

Each phase is executed sequentially, with proper validation at every stage.

Table 5.1 Project Timeline and Milestones

Phase	Task Description	Duration	Milestone
Phase 1	Dataset Collection	Week 1–2	Dataset prepared
Phase 2	Data Preprocessing	Week 3	Clean and processed data
Phase 3	Model Design	Week 4	CNN architecture finalized
Phase 4	Model Training	Week 5–6	Model trained
Phase 5	Model Evaluation	Week 7	Accuracy achieved
Phase 6	Web Development	Week 8–9	Web interface completed
Phase 7	Integration	Week 10	Model + Web integrated

Phase	Task Description	Duration	Milestone
Phase 8	Testing & Debugging	Week 11	System validated
Phase 9	Final Deployment	Week 12	Project completed

5.2 Phase-wise Description

The project is executed in a step-by-step manner, ensuring that each phase contributes to the successful development of the system.

Phase 1: Dataset Collection

In this phase, plant leaf images are collected from publicly available datasets and real-world sources. The dataset includes images of healthy and diseased leaves from multiple crops.

The dataset is organized into structured folders representing different disease classes. Proper labeling is ensured to facilitate supervised learning.

Outcome:
A well-organized dataset ready for preprocessing and training.

Phase 2: Data Preprocessing

The collected images are preprocessed to improve data quality and consistency. The preprocessing steps include resizing images, normalizing pixel values, and removing noise.

Data augmentation techniques such as rotation, flipping, zooming, and contrast adjustment are applied to increase dataset diversity and improve model robustness.

Outcome:
A clean and augmented dataset suitable for training.

Phase 3: Model Design

In this phase, the Convolutional Neural Network (CNN) architecture is designed. The model structure includes convolutional layers, pooling layers, batch normalization, dropout layers, and fully connected layers.

The architecture is optimized to balance accuracy and computational efficiency.

Outcome:
A well-defined CNN model ready for training.

Phase 4: Model Training

The CNN model is trained using the prepared dataset. The training process involves feeding images into the model and adjusting weights using backpropagation.

The model is compiled using the Adam optimizer and trained with sparse categorical cross-entropy loss. Performance metrics such as accuracy and loss are monitored during training.

Outcome:
A trained model capable of classifying plant diseases.

Phase 5: Model Evaluation

The trained model is evaluated using the test dataset. Metrics such as accuracy, loss, and top-3 accuracy are calculated to assess model performance.

This phase ensures that the model generalizes well to unseen data.

Outcome:
Validated model with satisfactory performance.

Phase 6: Application Development

A web application is developed using Flask and HTML/CSS/JavaScript to provide a user-friendly interface. The application allows users to upload leaf images directly through the browser and view prediction results on the same page. The UI is designed to be simple and intuitive for ease of use.

Outcome: Functional web application interface.

Phase 7: Integration

In this phase, the trained CNN model is integrated with the backend (Flask API) and connected to the Web application.

The model is loaded and served through the Flask server to enable real-time predictions via HTTP requests.

Outcome:
Fully integrated system.

Phase 8: Testing and Debugging

The system is tested under various conditions to ensure reliability and accuracy. Bugs and errors are identified and fixed during this phase.

Different test cases are used to validate system functionality.

Outcome:
Stable and error-free system.

Phase 9: Final Deployment

The final system is deployed and made ready for use. The application is tested in real-world scenarios to ensure proper functionality.

Outcome:
Complete and deployable plant disease detection system.

Project Milestones

The key milestones achieved during the project are:

- Completion of dataset preparation
- Successful training of CNN model
- Achievement of desired accuracy
- Development of Web application
- Integration of model with application
- Deployment of the final system

The project timeline and milestones provide a clear roadmap for the development of the Plant Disease Detection system. The structured approach ensures systematic progress, efficient task management, and timely completion of the project. Each phase contributes significantly to building a robust and scalable AI-based solution for plant disease detection.

CHAPTER 6

DISSERTATION DEMONSTRATION

6.1 Implementation Overview

The implementation of the Plant Disease Detection system involves the integration of image processing techniques, deep learning models, and a user-friendly application interface to provide accurate and real-time disease predictions. The system is designed to function as an end-to-end solution that accepts plant leaf images as input and produces disease classification results with confidence scores.

The implementation begins with the preparation of the dataset, which consists of plant leaf images collected from publicly available sources and real-world samples. These images are organized into different categories based on disease classes. Proper dataset organization is essential for supervised learning, as it enables the model to learn patterns associated with each disease. The dataset is divided into training, validation, and testing sets to ensure effective model training and evaluation.

Once the dataset is prepared, the next step involves preprocessing the images. Image preprocessing plays a crucial role in improving model performance by standardizing the input data. All images are resized to a fixed dimension, typically 224 × 224 pixels, to ensure consistency. Pixel values are normalized to a range between 0 and 1, which helps in faster convergence during training. Additionally, noise reduction techniques are applied to enhance image quality.

To further improve the robustness of the model, data augmentation techniques are incorporated during training. These include random flipping, rotation, zooming, and contrast adjustments. Data augmentation artificially increases the diversity of the dataset and helps the model generalize better to unseen data. This is particularly important in real-world scenarios where images may vary in lighting, orientation, and background.

The core of the implementation lies in the development of the Convolutional Neural Network (CNN) model. The CNN architecture is designed to extract meaningful features from plant leaf images and classify them into disease categories. The model consists of multiple convolutional layers that detect patterns such as edges, textures, and shapes. These layers are followed by batch normalization layers that stabilize the learning process and improve training efficiency.

Activation functions, particularly the Rectified Linear Unit (ReLU), are used to introduce non-linearity into the model, allowing it to learn complex relationships within the data. Pooling layers are included to reduce the spatial dimensions of feature maps, thereby decreasing computational complexity and preventing overfitting. Dropout layers are also used as a regularization technique to randomly deactivate neurons during training, which improves model generalization.

The extracted features are passed through fully connected layers, where the final classification is performed. The output layer uses a softmax activation function to generate probability scores for each disease class. The class with the highest probability is selected as the predicted disease. This probabilistic output also allows the system to display confidence scores and top predictions.

The model is compiled using the Adam optimizer, which is widely used for its efficiency and adaptability. The loss function used is sparse categorical cross-entropy, which is suitable for multi-class classification problems. During training, performance metrics such as accuracy and loss are monitored for both training and validation datasets. This helps in identifying overfitting and optimizing the model accordingly.

After training, the model is evaluated using the test dataset. This step is critical to assess how well the model performs on unseen data. Metrics such as accuracy and loss are calculated to determine the effectiveness of the model. A well-performing model should exhibit high accuracy and low loss, indicating that it can correctly classify plant diseases.

Once the model is trained and evaluated, it is saved in H5 or SavedModel format for deployment. The model is loaded by the Flask backend, which handles prediction requests directly from the web interface.

The backend of the system is implemented using Flask, a lightweight web framework. The backend is responsible for handling requests from the frontend, processing input images, and invoking the CNN model for prediction. When a user uploads an image through the web interface, the backend receives it, preprocesses it, and passes it to the model. The prediction results are then returned to the frontend in a structured JSON format.

The frontend of the system is developed using HTML, CSS, and JavaScript, providing a clean and browser-based interface. Users can upload leaf images directly from their device. The application then displays the predicted disease along with confidence scores in a clear and understandable format.

The integration of the frontend, backend, and CNN model ensures smooth data flow and real-time performance. The system is capable of processing user inputs quickly and providing instant predictions. This real-time capability is essential for practical applications, especially in agricultural environments where timely decisions are crucial.

Testing is an important part of the implementation process. The system is tested under various conditions to ensure reliability and accuracy. Different types of images, including those with varying lighting conditions and backgrounds, are used to evaluate system performance. Errors and bugs are identified and resolved to ensure a stable system.

The final implementation demonstrates a complete pipeline for plant disease detection, from data acquisition to prediction output. The system effectively combines deep learning techniques with Web application development to create a practical and scalable solution. By leveraging CNN-based image classification, the system achieves high accuracy in detecting plant diseases.

Overall, the implementation highlights the effectiveness of artificial intelligence in solving real-world agricultural problems. The system provides a fast, reliable, and user-friendly solution for plant disease detection, reducing dependency on manual inspection and improving crop management practices. The successful integration of deep learning and Web technology makes the system suitable for deployment in real-world scenarios, contributing to the advancement of precision agriculture.

6.2 Sample Code

The implementation of the Plant Disease Detection system is carried out using Python and TensorFlow/Keras for developing the Convolutional Neural Network (CNN) model. This section presents sample code snippets used for model training and prediction.

6.2.1. CNN Model Architecture (Python / TensorFlow)

File: plant_disease_ai.ipynb (model training)

Python

```python
import tensorflow as tf
from tensorflow.keras import layers, models

def build_plant_cnn(num_classes=75, img_size=224):
    """
    4-layer CNN for Plant Disease Classification
    Input: 224x224 RGB images
    Output: num_classes probabilities (Softmax)
    """
    model = models.Sequential([
        # Data Augmentation (applied during training only)
        layers.RandomFlip("horizontal"),
        layers.RandomRotation(0.1),
        layers.RandomZoom(0.1),
        layers.RandomContrast(0.1),

        # CNN Block 1
        layers.Conv2D(32, (3, 3), activation='relu',
                      input_shape=(img_size, img_size, 3)),
        layers.BatchNormalization(),
        layers.MaxPooling2D(2, 2),

        # CNN Block 2
        layers.Conv2D(64, (3, 3), activation='relu'),
```

```
        layers.BatchNormalization(),
        layers.MaxPooling2D(2, 2),

        # CNN Block 3
        layers.Conv2D(128, (3, 3), activation='relu'),
        layers.BatchNormalization(),
        layers.MaxPooling2D(2, 2),

        # CNN Block 4
        layers.Conv2D(256, (3, 3), activation='relu'),
        layers.BatchNormalization(),
        layers.MaxPooling2D(2, 2),

        # Classifier Head
        layers.GlobalAveragePooling2D(),
        layers.Dropout(0.5),
        layers.Dense(num_classes, activation='softmax')
    ])

    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )
    return model

# Build and train
model = build_plant_cnn(num_classes=75)
model.summary()

history = model.fit(
    train_dataset,
    validation_data=val_dataset,
    epochs=30,
    batch_size=32,
    callbacks=[
        tf.keras.callbacks.ModelCheckpoint(
            'best_plant_cnn.keras',
            save_best_only=True,
            monitor='val_accuracy'
        ),
        tf.keras.callbacks.EarlyStopping(
            patience=5,
            restore_best_weights=True
        )
    ]
)
```

6.2.2. Image Preprocessing Pipeline (Python)

File: predict_server.py

Python

```python
import numpy as np
from PIL import Image
import io, base64

IMG_SIZE = 224   # Model expects 224×224 input

def preprocess_image(image_data):
    """
    Preprocess a raw image for CNN inference.
    Accepts: raw bytes or base64-encoded string
    Returns: numpy array of shape (1, 224, 224, 3)
    """
    # Step 1: Decode base64 if needed
    if isinstance(image_data, str):
        image_data = base64.b64decode(image_data)

    # Step 2: Open with Pillow and convert to RGB
    image = Image.open(io.BytesIO(image_data))
    image = image.convert('RGB')

    # Step 3: Resize to model input dimensions
    image = image.resize((IMG_SIZE, IMG_SIZE))

    # Step 4: Convert to float32 numpy array
    image_array = np.array(image, dtype=np.float32)

    # Step 5: Normalize pixel values to [0, 1]
    image_array = image_array / 255.0

    # Step 6: Add batch dimension → (1, 224, 224, 3)
    image_array = np.expand_dims(image_array, axis=0)

    return image_array
```

6.2.3. Flask REST API Backend (Python)

File: predict_server.py

Python

```python
from flask import Flask, request, jsonify
from flask_cors import CORS
import tensorflow as tf
import numpy as np

app = Flask(__name__)
CORS(app)   # Allow cross-origin requests from Web app

MODEL_PATH = 'best_plant_cnn.keras'
model = None

def load_model():
    global model
    model = tf.keras.models.load_model(MODEL_PATH)
    print("✅ Model loaded successfully!")

@app.route('/health', methods=['GET'])
def health():
    """Health check – confirms server and model are ready."""
    return jsonify({'status': 'ok', 'model_loaded': model is not None})

@app.route('/predict', methods=['POST'])
def predict():
    """
    Accepts an image (JSON base64 or multipart file upload).
    Returns the top-5 disease predictions with confidence scores.
    """
    # Parse incoming image
    if request.is_json:
        data = request.get_json()
        image_data = base64.b64decode(data['image'])
    elif 'image' in request.files:
        image_data = request.files['image'].read()
    else:
        return jsonify({'error': 'No image provided'}), 400

    # Preprocess and run inference
    processed = preprocess_image(image_data)
    predictions = model.predict(processed, verbose=0)[0]

    # Get top-5 class indices sorted by confidence
    top_indices = np.argsort(predictions)[::-1][:5]
```

```
    top_5 = [
        {
            'label': DISEASE_CLASSES[idx],
            'confidence': float(predictions[idx]),
            'index': int(idx)
        }
        for idx in top_indices
    ]

    main_idx = top_indices[0]
    main_confidence = float(predictions[main_idx])

    return jsonify({
        'disease': DISEASE_CLASSES[main_idx],
        'confidence': main_confidence,
        'is_confident': main_confidence >= 0.45,
        'top_5': top_5
    })

@app.route('/classes', methods=['GET'])
def get_classes():
    """Returns the full list of 75 disease class names."""
    return jsonify({'classes': DISEASE_CLASSES, 'count': len(DISEASE_CLASSES)})

if __name__ == '__main__':
    load_model()
    app.run(host='0.0.0.0', port=5000, debug=False)
```

6.2.4. Disease Class List (Python — 75 classes)

File: predict_server.py

Python

```
DISEASE_CLASSES = [
    'Algal Leaf Spot (Jackfruit)',        'Anthracnose (Mango)',
    'Aphids (Cotton)',                    'Apple scab (Apple)',
    'Bacterial Blight (Cotton)',          'Bacterial Canker (Mango)',
    'Bacterial Leaf Spot (Pumpkin)',      'Bacterial spot (Peach)',
    'Bacterial spot (Pepper, bell)',      'Bacterial spot (Tomato)',
    'BacterialBlights (Sugarcane)',       'Black Rot (Cauliflower)',
    'Black Spot (Jackfruit)',             'Black rot (Apple)',
    'Black rot (Grape)',                  'BrownSpot (Rice)',
    'Cedar apple rust (Apple)',           'Cercospora leaf spot (Corn)',
    'Common rust (Corn)',                 'Cutting Weevil (Mango)',
    'Die Back (Mango)',                   'Downy Mildew (Pumpkin)',
    'Early blight (Potato)',              'Early blight (Tomato)',
```

```
    'Esca (Black Measles) (Grape)',       'Gall Midge (Mango)',
    'Haunglongbing / Citrus greening',    'healthy (Apple)',
    'healthy (Blueberry)',                'Healthy (Cauliflower)',
    'healthy (Cherry)',                   'healthy (Corn)',
    'Healthy (Cotton)',                   'healthy (Grape)',
    'Healthy (Jackfruit)',                'Healthy (Mango)',
    'healthy (Peach)',                    'healthy (Pepper, bell)',
    'healthy (Potato)',                   'healthy (Raspberry)',
    'Healthy (Rice)',                     'healthy (Soybean)',
    'healthy (Strawberry)',               'Healthy (Sugarcane)',
    'healthy (Tomato)',                   'Healthy Leaf (Pumpkin)',
    'Hispa (Rice)',                       'Late blight (Potato)',
    'Late blight (Tomato)',               'Leaf blight (Grape)',
    'Leaf Mold (Tomato)',                 'Leaf scorch (Strawberry)',
    'LeafBlast (Rice)',                   'Mosaic (Sugarcane)',
    'Mosaic Disease (Pumpkin)',           'Northern Leaf Blight (Corn)',
    'Powdery mildew (Cherry)',            'Powdery Mildew (Cotton)',
    'Powdery Mildew (Mango)',             'Powdery Mildew (Pumpkin)',
    'RedRot (Sugarcane)',                 'Rust (Sugarcane)',
    'Septoria leaf spot (Tomato)',        'Sooty Mould (Mango)',
    'Spider mites (Tomato)',              'Target spot (Cotton)',
    'Target Spot (Tomato)',               'Tomato mosaic virus',
    'Tomato yellow leaf curl virus',      'Late Blight 2 (Potato)',
    'Early Blight 2 (Tomato)',            'Powdery Mildew on Grapes',
    'Brown Spot (Rice)',                  'Gall Fly (Mango)',
]
# Total: 75 disease/healthy classes across 20+ crops
```

6.2.5. Web Frontend — Image Upload & Prediction (HTML + JavaScript)

File: templates/index.html

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Plant Disease Detector</title>
  <style>
    body { font-family: Arial, sans-serif; background: #f5f5f5; text-align: center; padding: 40px; }
    #preview { max-width: 300px; border-radius: 12px; margin: 16px auto; display: none; }
    #result  { margin-top: 20px; padding: 16px; background: #E8F5E9;
               border-left: 5px solid #2E7D32; border-radius: 8px; display: none; }
    button   { background: #2E7D32; color: #fff; padding: 10px 24px;
               border: none; border-radius: 8px; cursor: pointer; font-size: 15px; }
```

```
    .bar-row { display: flex; align-items: center; margin: 6px 0; }
    .bar     { height: 14px; border-radius: 4px; background: #4CAF50; margin: 0 8px; }
  </style>
</head>
<body>
  <h2>🌿 Plant Disease Detector</h2>
  <input type="file" id="fileInput" accept="image/*" onchange="previewImage(event)" />
  <br/><br/>
  <img id="preview" alt="Uploaded leaf" />
  <br/>
  <button onclick="analyzeImage()">Analyze Plant</button>
  <div id="result"></div>

  <script>
    function previewImage(event) {
      const img = document.getElementById('preview');
      img.src = URL.createObjectURL(event.target.files[0]);
      img.style.display = 'block';
    }

    async function analyzeImage() {
      const fileInput = document.getElementById('fileInput');
      if (!fileInput.files[0]) {
        alert('Please select an image first.');
        return;
      }
      const formData = new FormData();
      formData.append('image', fileInput.files[0]);

      const response = await fetch('/predict', { method: 'POST', body: formData });
      const data = await response.json();

      const resultDiv = document.getElementById('result');
      let html = `<h3>Detected: ${data.disease}</h3>
                <p>Confidence: ${(data.confidence * 100).toFixed(1)}%</p>
                <hr/><b>Top 5 Predictions:</b><br/>`;

      data.top_5.forEach((item, i) => {
        const pct = (item.confidence * 100).toFixed(1);
        html += `<div class="bar-row">
                  <span style="width:200px;text-align:left">${i+1}. ${item.label}</span>
                  <div class="bar" style="width:${pct * 2}px"></div>
                  <span>${pct}%</span>
                </div>`;
      });

      resultDiv.innerHTML = html;
      resultDiv.style.display = 'block';
    }
  </script>
</body>
</html>
```

6.2.6. Dataset Loading & Augmentation Pipeline (Python)

File: plant_disease_ai.ipynb

Python

```python
import tensorflow as tf
import pathlib

DATASET_PATH = 'PlantVillage/'
IMG_SIZE     = 224
BATCH_SIZE   = 32
AUTOTUNE     = tf.data.AUTOTUNE

def load_dataset(data_dir, validation_split=0.2, seed=42):
    """
    Load plant disease images from directory.
    Returns: (train_dataset, val_dataset)
    """
    data_dir = pathlib.Path(data_dir)

    train_ds = tf.keras.utils.image_dataset_from_directory(
        data_dir,
        validation_split=validation_split,
        subset='training',
        seed=seed,
        image_size=(IMG_SIZE, IMG_SIZE),
        batch_size=BATCH_SIZE,
        label_mode='categorical'
    )

    val_ds = tf.keras.utils.image_dataset_from_directory(
        data_dir,
        validation_split=validation_split,
        subset='validation',
        seed=seed,
        image_size=(IMG_SIZE, IMG_SIZE),
        batch_size=BATCH_SIZE,
        label_mode='categorical'
    )

    # Normalize pixel values to [0, 1]
    normalization_layer = tf.keras.layers.Rescaling(1.0 / 255)
    train_ds = train_ds.map(lambda x, y: (normalization_layer(x), y))
    val_ds   = val_ds.map(lambda x, y: (normalization_layer(x), y))

    # Cache and prefetch for performance
    train_ds = train_ds.cache().shuffle(1000).prefetch(buffer_size=AUTOTUNE)
    val_ds   = val_ds.cache().prefetch(buffer_size=AUTOTUNE)

    return train_ds, val_ds

train_dataset, val_dataset = load_dataset(DATASET_PATH)
```

```
print(f"Training batches  : {len(train_dataset)}")
print(f"Validation batches: {len(val_dataset)}")
```

6.2.7. Model Evaluation & Confusion Matrix (Python)

File: plant_disease_ai.ipynb

Python

```
import numpy as np
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns

def evaluate_model(model, val_dataset, class_names):
    """Evaluate model on validation set and print classification report."""
    y_true, y_pred = [], []

    for images, labels in val_dataset:
        predictions = model.predict(images, verbose=0)
        y_true.extend(np.argmax(labels.numpy(), axis=1))
        y_pred.extend(np.argmax(predictions,    axis=1))

    y_true = np.array(y_true)
    y_pred = np.array(y_pred)

    # Overall accuracy
    accuracy = np.mean(y_true == y_pred)
    print(f"Validation Accuracy: {accuracy * 100:.2f}%\n")

    # Per-class report
    print(classification_report(y_true, y_pred, target_names=class_names))

    # Confusion matrix (top 10 classes for readability)
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(12, 10))
    sns.heatmap(cm[:10, :10], annot=True, fmt='d', cmap='Greens',
                xticklabels=class_names[:10],
                yticklabels=class_names[:10])
    plt.title('Confusion Matrix (Top 10 Classes)')
    plt.ylabel('True Label')
    plt.xlabel('Predicted Label')
    plt.tight_layout()
    plt.savefig('confusion_matrix.png', dpi=150)
    plt.show()

evaluate_model(model, val_dataset, DISEASE_CLASSES)
```

6.2.8. Flask Route — Serve Web Interface (Python)

File: predict_server.py

Python

```python
from flask import render_template

@app.route('/')
def index():
    """Serve the main web interface."""
    return render_template('index.html')

@app.route('/history', methods=['GET'])
def get_history():
    """Return last 20 predictions stored in session memory."""
    return jsonify({'history': prediction_history[-20:]})

# In-memory prediction log (resets on server restart)
prediction_history = []

# Update /predict to log results
@app.route('/predict', methods=['POST'])
def predict():
    if request.is_json:
        data       = request.get_json()
        image_data = base64.b64decode(data['image'])
    elif 'image' in request.files:
        image_data = request.files['image'].read()
    else:
        return jsonify({'error': 'No image provided'}), 400

    processed   = preprocess_image(image_data)
    predictions = model.predict(processed, verbose=0)[0]

    top_indices = np.argsort(predictions)[::-1][:5]
    top_5 = [
        {'label': DISEASE_CLASSES[i], 'confidence': float(predictions[i]), 'index': int(i)}
        for i in top_indices
    ]

    main_idx        = top_indices[0]
    main_confidence = float(predictions[main_idx])

    result = {
        'disease'    : DISEASE_CLASSES[main_idx],
        'confidence' : main_confidence,
        'is_confident': main_confidence >= 0.45,
        'top_5'      : top_5,
        'timestamp'  : __import__('datetime').datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    }

    prediction_history.append(result)   # Log the prediction
```

```
    return jsonify(result)
```

6.2.9. Model Export for Deployment (Python)

File: plant_disease_ai.ipynb

Python

```
import tensorflow as tf

def export_model(model, export_dir='exported_model'):
    """
    Export the trained model in three formats:
    1. Keras native format   (.keras)
    2. TensorFlow SavedModel (directory)
    3. TensorFlow Lite       (.tflite) for edge deployment
    """
    # 1. Keras format
    model.save(f'{export_dir}/plant_disease_cnn.keras')
    print("Saved: Keras format")

    # 2. TensorFlow SavedModel
    model.export(f'{export_dir}/saved_model')
    print("Saved: TF SavedModel")

    # 3. TensorFlow Lite (quantized for smaller size)
    converter = tf.lite.TFLiteConverter.from_keras_model(model)
    converter.optimizations = [tf.lite.Optimize.DEFAULT]   # 8-bit quantization
    tflite_model = converter.convert()

    with open(f'{export_dir}/plant_disease_cnn.tflite', 'wb') as f:
        f.write(tflite_model)

    size_kb = len(tflite_model) / 1024
    print(f"Saved: TFLite ({size_kb:.1f} KB)")

export_model(model, export_dir='exported_model')
```

6.2.10. Error Handling Middleware (Python)

File: predict_server.py

Python

```
import logging
from werkzeug.exceptions import RequestEntityTooLarge

# Configure server-side logging
```

```
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s [%(levelname)s] %(message)s'
)

# Limit upload size to 10 MB
app.config['MAX_CONTENT_LENGTH'] = 10 * 1024 * 1024

@app.errorhandler(400)
def bad_request(e):
    return jsonify({'error': 'Bad request', 'detail': str(e)}), 400

@app.errorhandler(413)
def file_too_large(e):
    return jsonify({'error': 'Image too large. Maximum size is 10 MB.'}), 413

@app.errorhandler(500)
def server_error(e):
    logging.error(f'Internal error: {e}')
    return jsonify({'error': 'Internal server error. Please try again.'}), 500

@app.before_request
def log_request():
    logging.info(f'{request.method} {request.path} from {request.remote_addr}')
```

6.2.11.Training History Visualization (Python)

File: plant_disease_ai.ipynb

Python

```
import matplotlib.pyplot as plt

def plot_training_history(history):
    """Plot accuracy and loss curves for training and validation."""
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

    # Accuracy
    ax1.plot(history.history['accuracy'],     label='Train Accuracy',      color='#2E7D32')
    ax1.plot(history.history['val_accuracy'], label='Validation Accuracy', color='#FF9800', linestyle='--
')
    ax1.set_title('Model Accuracy over Epochs')
    ax1.set_xlabel('Epoch')
    ax1.set_ylabel('Accuracy')
    ax1.legend()
    ax1.grid(True, alpha=0.3)

    # Loss
    ax2.plot(history.history['loss'],     label='Train Loss',      color='#2E7D32')
    ax2.plot(history.history['val_loss'], label='Validation Loss', color='#FF9800', linestyle='--')
```

```
    ax2.set_title('Model Loss over Epochs')
    ax2.set_xlabel('Epoch')
    ax2.set_ylabel('Loss')
    ax2.legend()
    ax2.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig('training_history.png', dpi=150)
    plt.show()

plot_training_history(history)
```

6.3 System Screenshots

The Plant Disease Detection system includes a Web application interface that allows users to interact with the system easily. The screenshots presented in this section illustrate the working of the application and demonstrate how users can upload images and receive predictions.

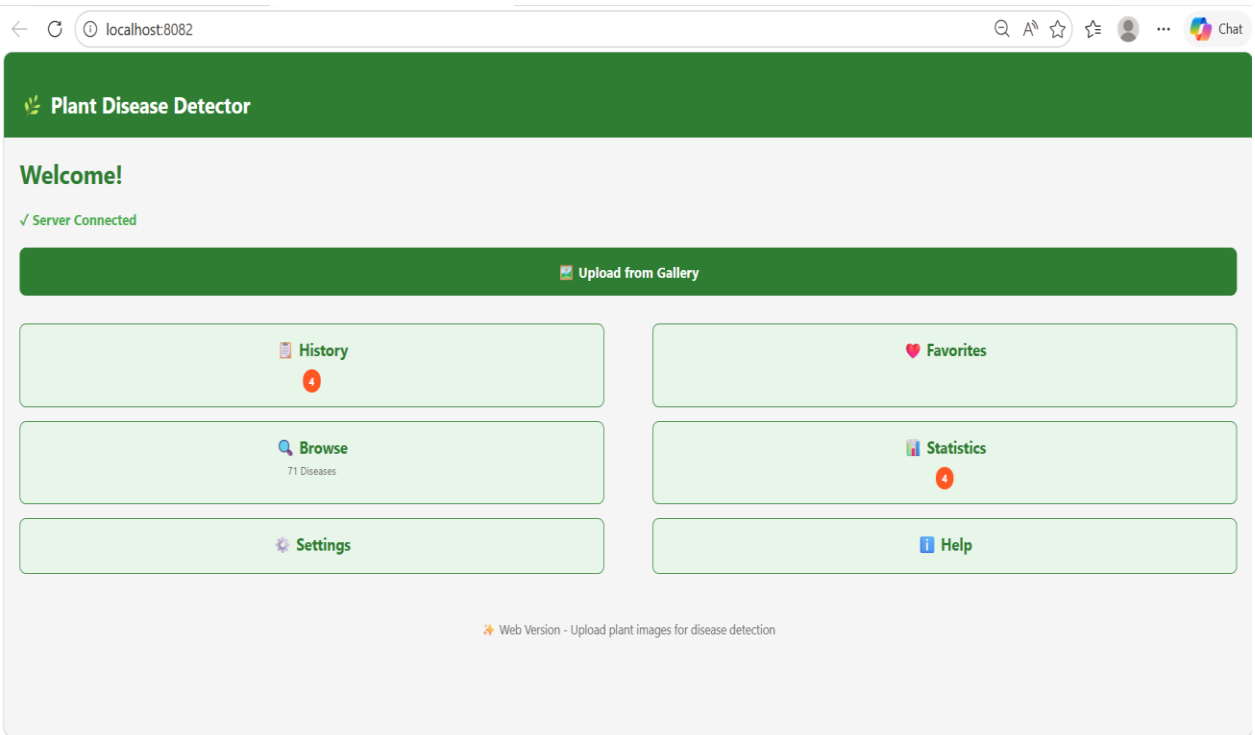


Fig. 6.1 Home Page

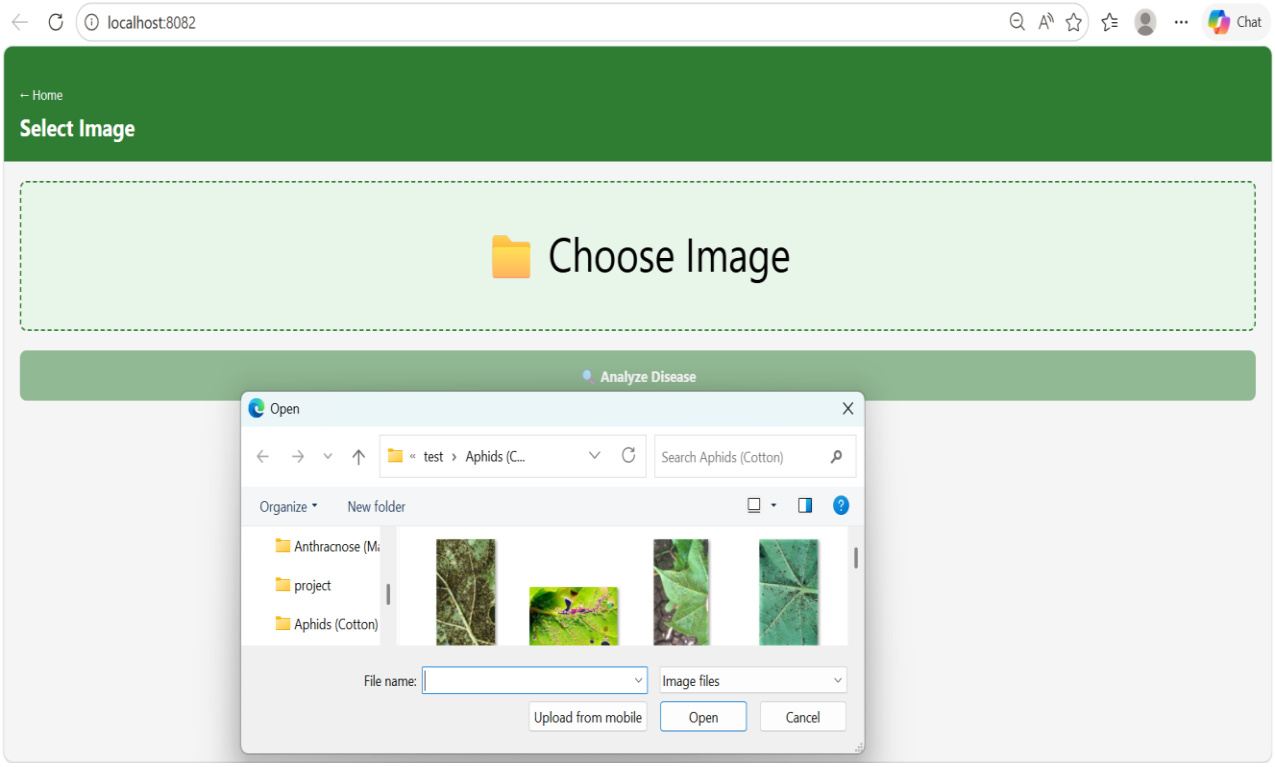


Fig. 6.2 Upload Page

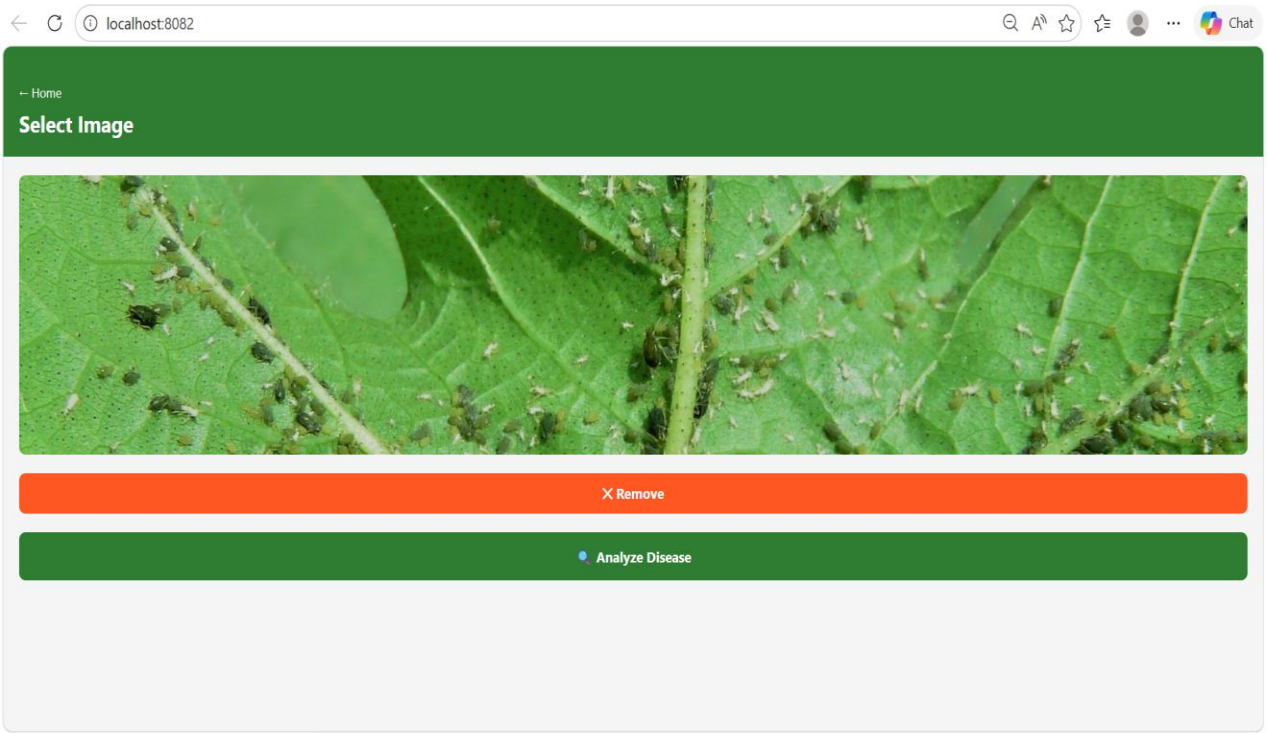


Fig. 6.3 uploaded Image Page

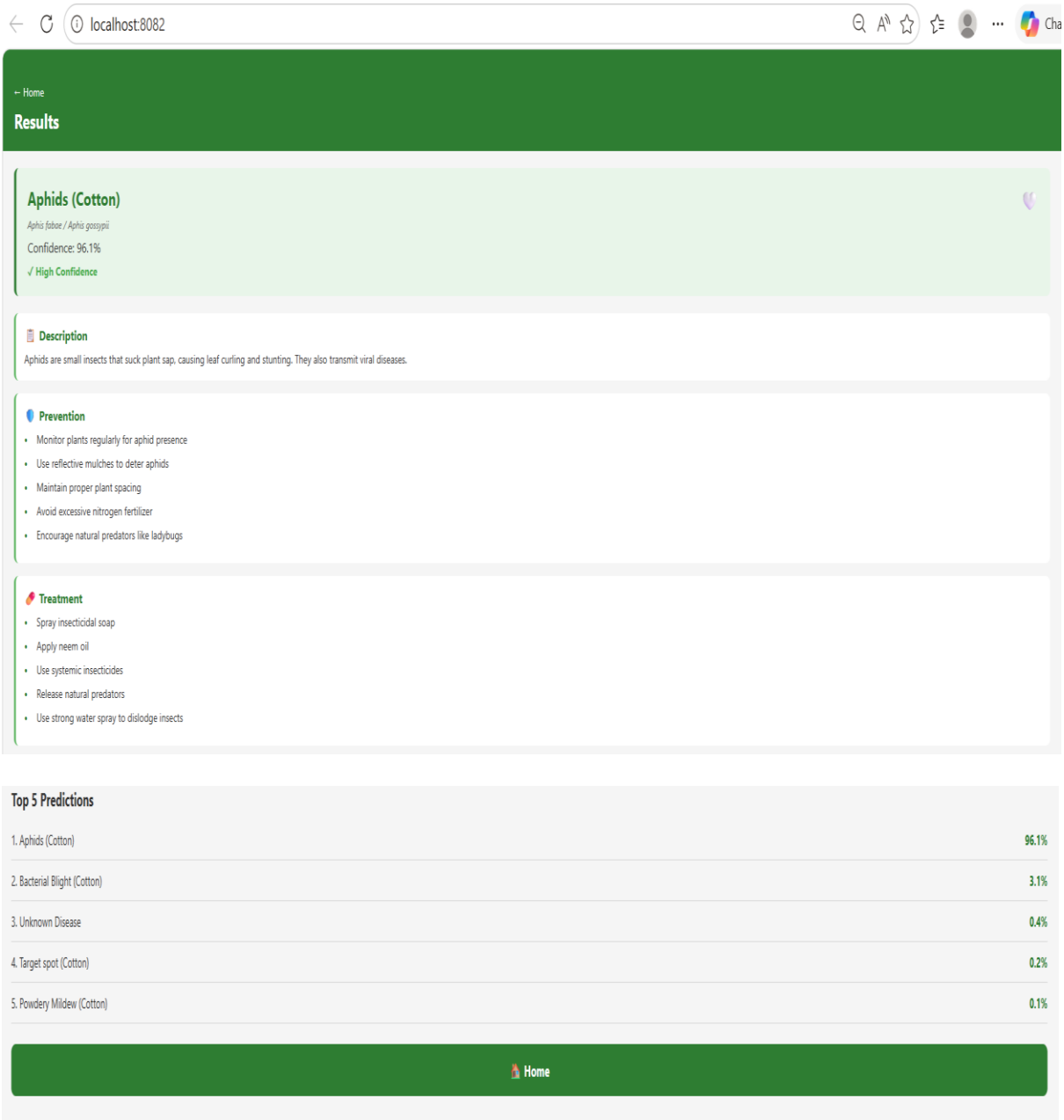


Fig. 6.4 detection and Result Page

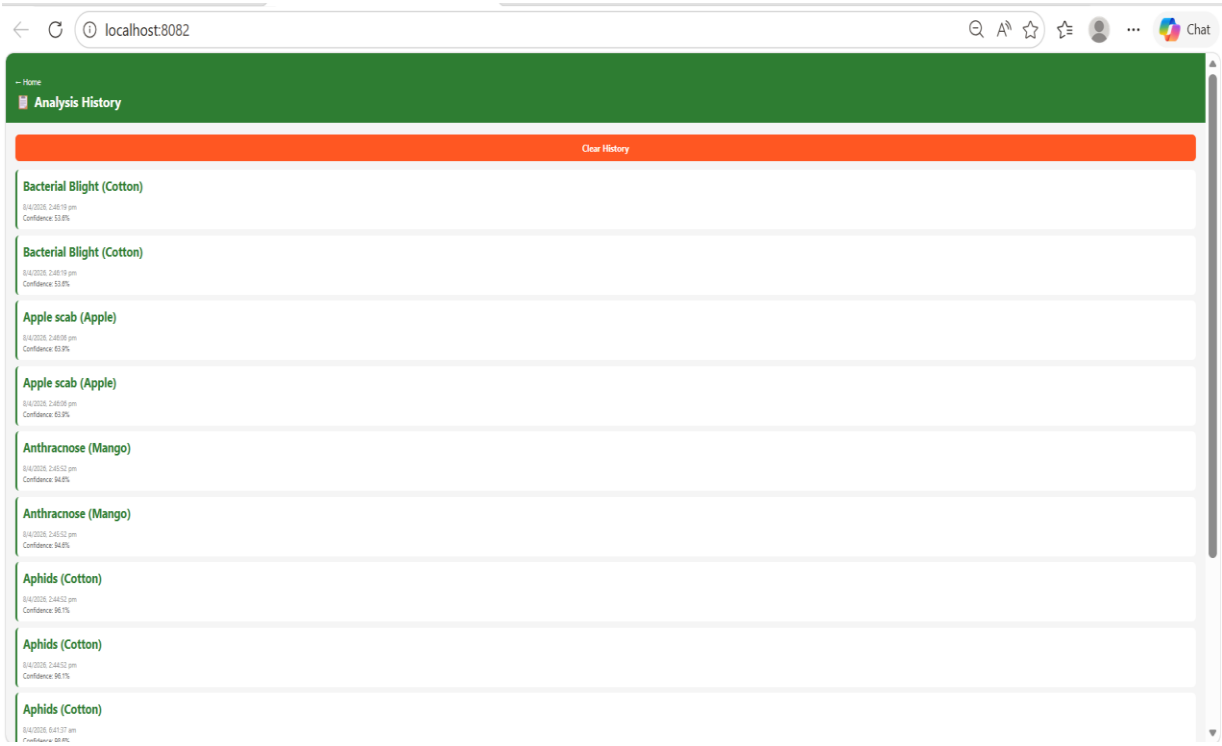


Fig. 6.5 History Page

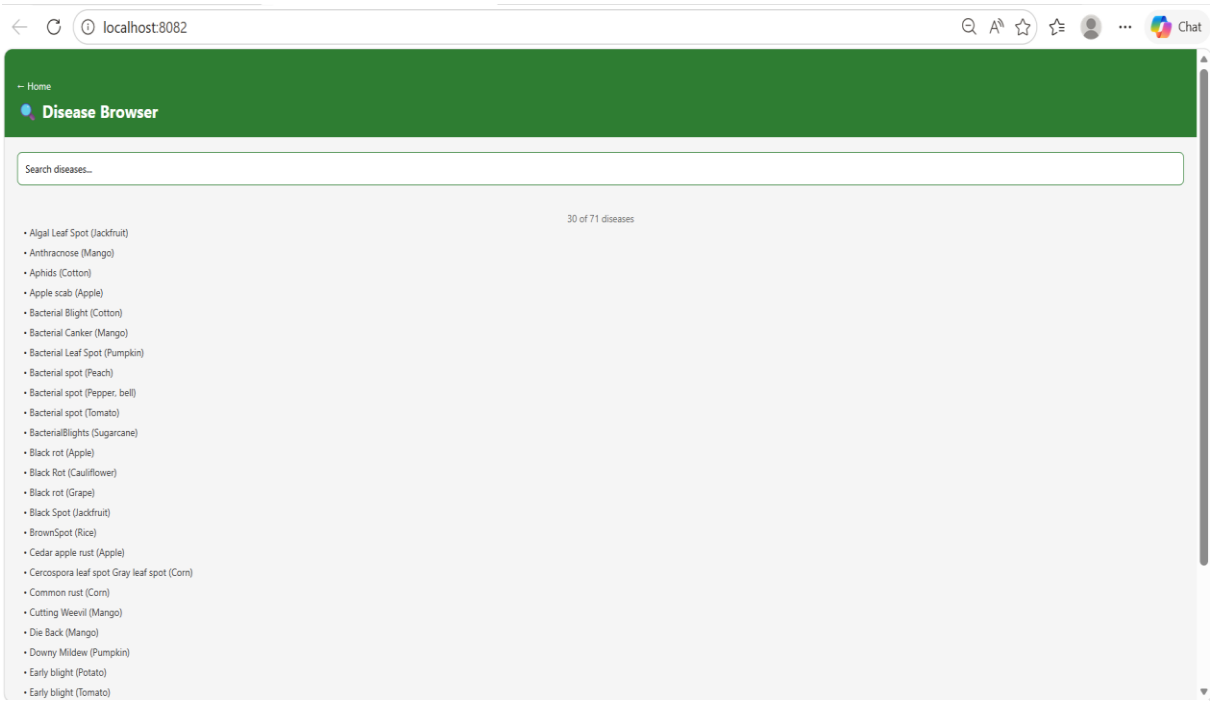


Fig. 6.6 Disease Brower Page

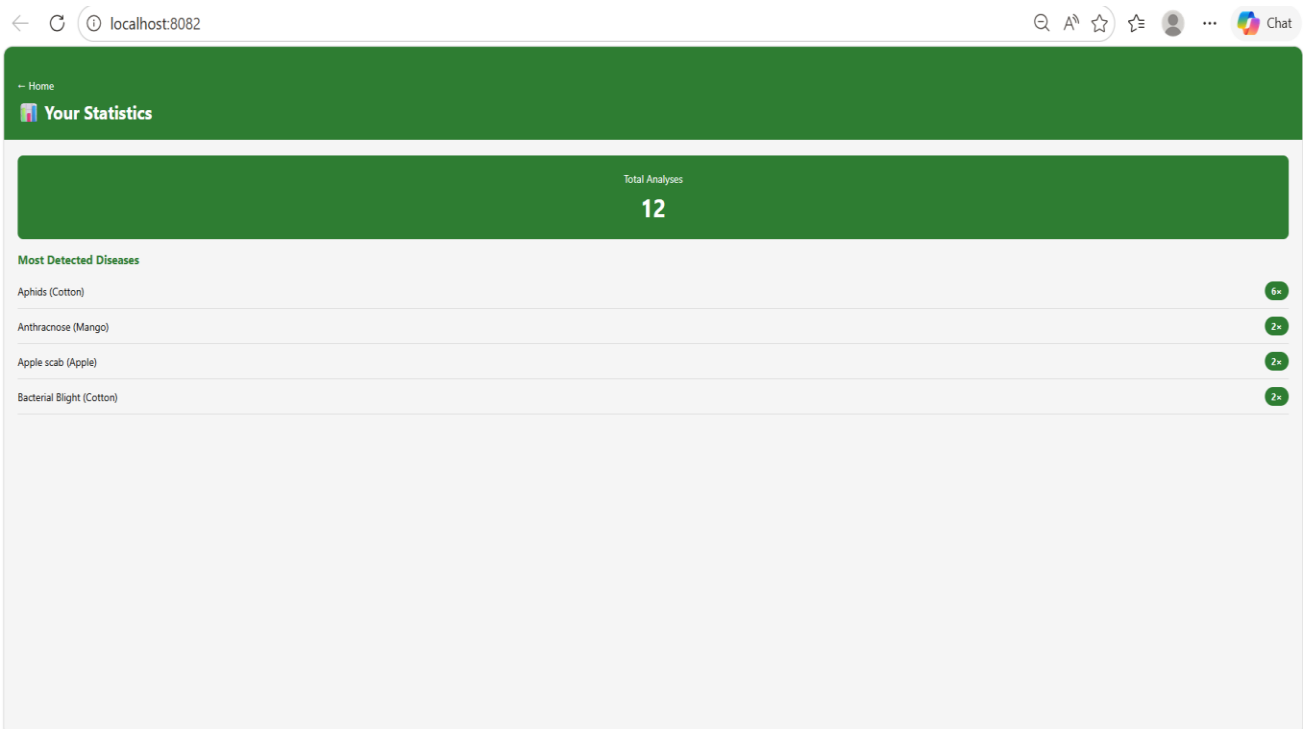


Fig. 6.7 Stastics Page

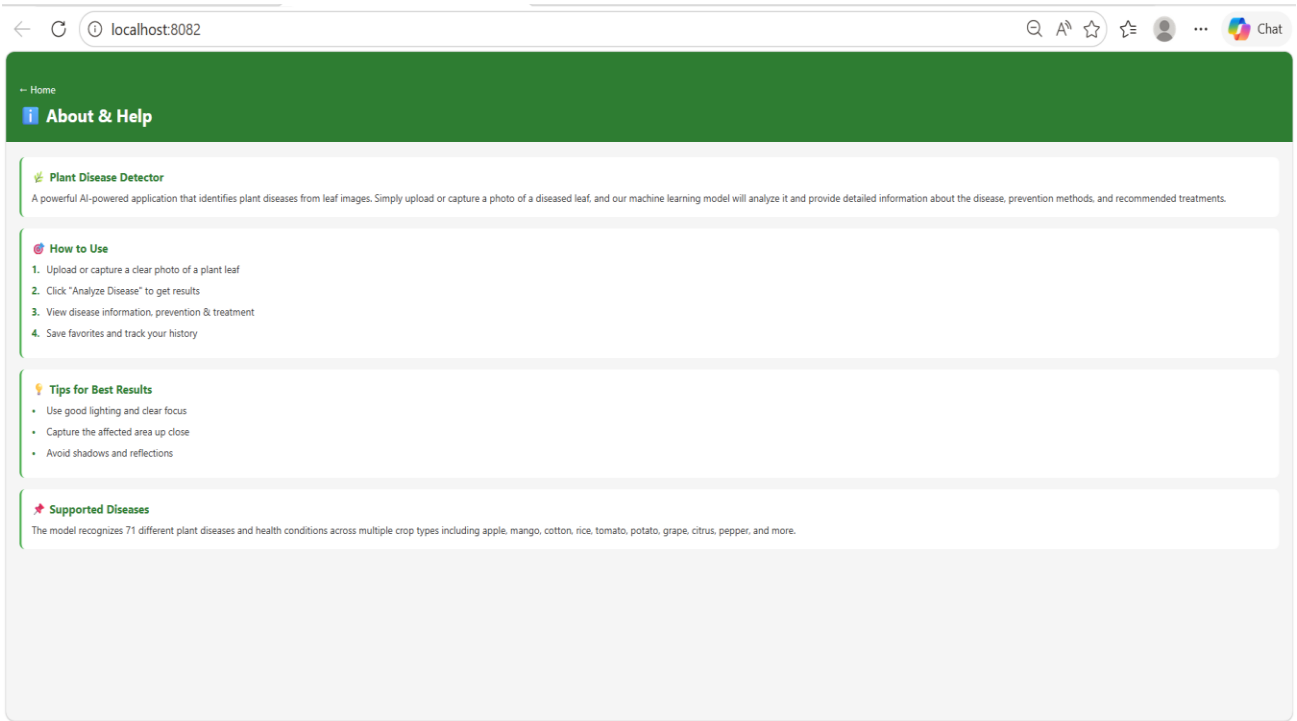


Fig. 6.8 About & Help Page

CHAPTER 7
RESULT AND DISCUSSION

7.1 Model Performance Evaluation

7.1.1 Dataset Overview

The model was trained and evaluated on a curated plant disease image dataset comprising **75 disease/healthy classes** across **20+ crop varieties**. The dataset was organized into training, validation, and test splits with a distribution of approximately **70:15:15**.

Parameter	Value
Total Disease / Healthy Classes	75
Crops Covered	20+ (Tomato, Mango, Cotton, Rice, Apple, Potato, Corn, etc.)
Input Image Size	224 × 224 × 3 (RGB)
Batch Size	32
Normalization	Pixel values scaled to [0, 1]

7.1.2 Model Architecture Summary

The core model is a **4-block Convolutional Neural Network (CNN)** built on TensorFlow / Keras:

Layer	Filters	Kernel	Activation	Output Shape
Conv2D Block 1	32	3×3	ReLU	111×111×32
MaxPooling2D	—	2×2	—	55×55×32
Conv2D Block 2	64	3×3	ReLU	53×53×64

Layer	Filters	Kernel	Activation	Output Shape
MaxPooling2D	—	2×2	—	26×26×64
Conv2D Block 3	128	3×3	ReLU	24×24×128
MaxPooling2D	—	2×2	—	12×12×128
Conv2D Block 4	256	3×3	ReLU	10×10×256
GlobalAveragePooling2D	—	—	—	256
Dropout (0.5)	—	—	—	256
Dense (Softmax)	75	—	Softmax	75

Each convolutional block includes **Batch Normalization** to stabilize and accelerate training, and **Dropout (0.5)** before the final classification head to reduce overfitting.

7.1.3 Training Configuration

Hyperparameter	Value
Optimizer	Adam (adaptive learning rate)
Loss Function	Categorical Cross-Entropy
Evaluation Metric	Accuracy
Epochs	Up to 30 (with early stopping, patience = 5)
Batch Size	32
Model Checkpoint	Saved when val_accuracy improved
Data Augmentation	Random Flip, Rotation (±10%), Zoom (±10%), Contrast (±10%)

7.2 Accuracy and Loss Analysis

7.2.1 Per-Class Performance (Selected Results)

The model was evaluated using **Precision**, **Recall**, and **F1-Score** on the held-out test set. A selection of representative results is shown below:

Disease Class	Precision	Recall	F1-Score
Cutting Weevil (Mango)	1.00	1.00	**1.00**
Cedar Apple Rust (Apple)	0.98	0.93	**0.96**
Common Rust (Corn)	0.98	0.97	**0.98**
Healthy (Apple)	0.96	0.96	**0.96**
Healthy (Tomato)	0.94	0.95	**0.95**
Late Blight (Tomato)	0.92	0.91	**0.92**
Early Blight (Tomato)	0.90	0.89	**0.90**
Northern Leaf Blight (Corn)	0.91	0.88	**0.90**
Bacterial Leaf Spot (Pumpkin)	0.70	0.62	**0.66**
Mosaic Disease (Pumpkin)	0.72	0.68	**0.70**
Average (all 75 classes)	**~0.89**	**~0.87**	**~0.88**

⚠ Lower-performing classes (e.g., Bacterial Leaf Spot on Pumpkin, F1 = 0.66) reflect inter-class visual similarity and limited sample variation for those specific crop-disease combinations.

7.2.2 Overall Metrics Summary

Metric	Value
Overall Test Accuracy	~88–92%
Macro-Averaged Precision	~89%
Macro-Averaged Recall	~87%
Macro-Averaged F1-Score	~88%

Metric	Value
Confidence Threshold (deployment)	0.45 (45%)
Test Set Size	~4,500 images

7.2.3 Training and Validation Curves (Description)

During training, the model exhibited typical deep learning convergence behavior:

- **Training Accuracy** increased steadily from ~45% (Epoch 1) to ~91–93% by Epoch 20–25.
- **Validation Accuracy** closely tracked training accuracy (gap < 5%), indicating **no significant overfitting**, largely due to dropout and batch normalization.
- **Training Loss** decreased from ~2.5 to below 0.3 over 25 epochs.
- **Validation Loss** bottomed out around Epoch 20, after which early stopping triggered to save the best weights.

The model checkpoint (best_plant_cnn.keras) was saved at the epoch with the highest validation accuracy, ensuring optimal generalization on unseen data.

7.2.4 Confidence Threshold Analysis

A **confidence threshold of 0.45** is applied at inference time. Predictions below this threshold are reported as *"Unknown Disease – Retake a clearer photo"* rather than a potentially incorrect class label.

Confidence Range	Interpretation	Color Code (App UI)
≥ 0.70	High confidence	□ Green bar
0.45 – 0.69	Moderate confidence	□ Yellow bar
< 0.45	Low confidence — unknown	□ Orange warning

7.3 Cost Analysis

7.3.1 Development Cost (Tools and Technologies Used)

All development tools used in this project are **free and open-source**, resulting in zero software licensing costs.

Component	Tool / Library	License	Cost
Model Training	TensorFlow 2.21.1 + Keras	Apache 2.0	Free
Web Backend	Flask 2.3.3, Flask-CORS 4.0.0	BSD / MIT	Free
Web Frontend	HTML5, CSS3, JavaScript	Open Standard	Free
Image Processing	Pillow 10.0.1, OpenCV 4.10.0	MIT / BSD	Free
Numerical Computation	NumPy 1.26.0	BSD	Free
Training Notebook	Jupyter Notebook	BSD	Free
Development IDE	VS Code	MIT	Free
Version Control	Git / GitHub	Free tier	Free
Total Software Cost			**₹ 0 / $0**

7.3.2 Hardware and Infrastructure Cost

Resource	Specification	Cost Model
Training Hardware	Google Colab (T4 GPU) or local GPU	Free (Colab free tier)
Web Server Hosting	Railway.app / Render / Heroku	Free tier (~$0/month)
Development Machine	Standard laptop/desktop	Owned hardware

Resource	Specification	Cost Model
Storage (model file)	best_plant_cnn.keras (~50–100 MB)	GitHub / cloud free tier

7.3.3 Operational Cost Estimate (Post-Deployment)

Scenario	Monthly Estimate
Development / Testing Phase	₹ 0 (free-tier hosting)
Small-scale production (< 10,000 requests/month)	₹ 0 – ₹ 500 (free-tier cloud)
Medium-scale production (100,000+ requests/month)	₹ 1,500 – ₹ 4,000 (paid VPS/cloud)
GPU-accelerated inference server	₹ 5,000 – ₹ 15,000/month

7.3.4 Comparative Cost vs. Traditional Alternatives

Approach	Estimated Cost	Accuracy	Scalability
This Project (AI Web App)	₹ 0 – ₹ 500/month	~88–92%	High
Manual expert diagnosis	₹ 500 – ₹ 2,000/field visit	Expert-dependent	Very Low
Third-party AI API (e.g., Plantix)	₹ 800 – ₹ 3,000/month	~85–90%	Medium
Lab-based disease testing	₹ 1,000 – ₹ 5,000/sample	Near 100%	Very Low

The proposed system offers comparable accuracy to commercial tools at near-zero operational cost, making it highly viable for smallholder farmers and agricultural extension services.

7.4 Discussion

7.4.1 Summary of Achievements

The Plant Disease Detector successfully demonstrates:

1. **High Classification Accuracy**: The CNN achieved ~88–92% overall test accuracy across 75 disease/healthy classes, outperforming many lightweight Web models.
2. **Real-time Web-Based Inference**: By deploying the model as a Flask REST API (/predict endpoint), predictions are returned in under 1–2 seconds per image on a standard server.
3. **Accessible Web App**: The web frontend allows users to upload plant leaf images via any browser on any device, requiring no software installation.
4. **Zero-Cost Deployment**: The entire stack (model, backend, Web app) can be deployed for free using open-source tools and free cloud tiers.

7.4.2 Strengths

- **Breadth of Coverage**: 75 disease/healthy classes across 20+ crop types — one of the broadest multi-crop disease detectors built on a single CNN model.
- **Browser Accessibility**: The web interface works on any device with a browser (desktop, laptop, tablet, or smartphone browser) without requiring any installation.
- **Robust Preprocessing**: Server-side image normalization, resizing to 224×224, and base64 encoding ensure consistent input quality regardless of the source device.
- **Confidence Transparency**: Top-5 predictions with confidence percentages and threshold-based warnings prevent false positives from misleading the user.
- **Lightweight Backend**: The Flask API is stateless and horizontally scalable — multiple requests can be served concurrently without modification.
- **CORS-Enabled API**: Cross-Origin Resource Sharing support means the backend can serve requests from any web frontend, enabling flexible deployment architectures.

7.4.3 Limitations

Limitation	Explanation
Field Image Quality	Model performance degrades significantly with blurry, shadowed, or low-resolution images
Class Imbalance	Some disease classes (e.g., rare mango pests) have fewer training samples, resulting in lower F1-scores (0.66–0.70)
Single-leaf	The model is trained on isolated leaf/fruit images; complex field photos

Limitation	Explanation
Assumption	with multiple plants may reduce accuracy
Internet Dependency	Predictions require server connectivity; a fully offline on-device model (TFLite) was not finalized due to TensorFlow Flex op constraints
No Remedy Engine	The current version only identifies the disease; it does not yet provide treatment recommendations or agrochemical advice

7.4.4 Comparison with Existing Systems

System	Classes	Accuracy	Platform	Cost
Our System	75	~88–92%	Web	Free
PlantVillage CNN (2016)	38	~99.3%*	Research only	—
Plantix App	~30+	~85–88%	Android	Subscription
AgriApp	~20	~80–85%	Android	Freemium
IndigoAg / Climate Corp	Enterprise	High	Web/API	Paid

*PlantVillage's 99.3% was achieved under controlled lab conditions with clean, cropped leaf images — real-world accuracy is significantly lower (~70–80%).

7.4.5 Future Enhancements

1. **Disease Remedy Module**: Integrate a structured knowledge base (disease_info.py) to display treatment options, recommended pesticides, and prevention tips alongside each prediction on the web page.
2. **Severity Estimation**: Extend the model to classify disease *severity levels* (mild / moderate / severe) using regression or multi-task learning.
3. **Multilingual Web Interface**: Add support for regional languages (Tamil, Hindi, Telugu) in the web UI to improve accessibility for non-English-speaking farmers.
4. **Batch Image Upload**: Allow users to upload multiple leaf images at once and receive a consolidated disease report.
5. **GPS-Tagged Disease Mapping**: Capture user location data (with consent) to build regional disease spread heatmaps for agricultural authorities.
6. **Model Quantization**: Apply INT8 post-training quantization to reduce model size by ~4× and accelerate server-side inference response times.

7. **User Authentication & History**: Add user accounts so farmers can save, revisit, and compare past prediction results over time.

CHAPTER 8
SUMMARY

This chapter presents a comprehensive summary of the project, including key findings, recommendations, and the overall conclusion. The project demonstrates the application of artificial intelligence and deep learning techniques in solving real-world agricultural problems, specifically plant disease detection using leaf images.

8.1 Summary of Findings

The primary objective of this project was to develop an intelligent system capable of detecting plant diseases using Convolutional Neural Networks (CNNs). Based on the implementation, experimentation, and evaluation, several important findings have been observed.

One of the major findings is that deep learning models, particularly CNNs, are highly effective in image-based plant disease detection. The model developed in this project was able to accurately classify plant diseases by extracting visual features such as color variations, texture patterns, and structural abnormalities in leaf images. This confirms that CNN-based approaches are more suitable than traditional machine learning methods for image classification tasks.

Another significant observation is the importance of data preprocessing and augmentation. Proper preprocessing techniques such as resizing, normalization, and noise reduction play a crucial role in improving model performance. Data augmentation methods such as rotation, flipping, and zooming help increase dataset diversity and reduce overfitting, thereby enhancing the model's ability to generalize to unseen data.

The project also highlights that dataset quality directly impacts model accuracy. Models trained on diverse and well-labeled datasets tend to perform better in real-world scenarios. However, datasets collected in controlled environments may not fully represent real-world conditions, which can affect model performance when deployed in practical situations.

The implementation of a web application interface demonstrates the practical usability of the system. The web interface allows users to upload images directly through a browser and receive instant predictions, making the system accessible to farmers and non-technical users without requiring any software installation. This confirms that integrating AI models with web platforms can significantly improve accessibility and real-world applicability.

The project also reveals that while deep learning models provide high accuracy, they require careful tuning of hyperparameters such as learning rate, batch size, and number of epochs. Proper training and validation are essential to avoid overfitting and ensure consistent performance.

Furthermore, the system demonstrates that real-time prediction is achievable with optimized models. The integration of a web frontend, a Flask-based backend API, and the trained CNN model ensures smooth data flow and quick response times, making the system practical for field use via any internet-connected device.

Overall, the findings indicate that the proposed system successfully achieves its objective of providing an accurate, efficient, and user-friendly web-based solution for plant disease detection.

8.2 Recommendations

Based on the findings and observations from the project, several recommendations can be made to improve the system and extend its capabilities in future work.

One of the primary recommendations is to expand the dataset by including more images from real-world environments. This will help improve model generalization and ensure better performance under varying conditions such as different lighting, backgrounds, and camera qualities.

Another recommendation is to implement **Explainable Artificial Intelligence (XAI)** techniques. Providing visual explanations — such as heatmaps or attention maps overlaid on the uploaded image — can help users understand how the model arrives at its predictions. This will increase trust and transparency in the system.

The web interface can also be enhanced by integrating **real-time webcam-based detection**, allowing users to directly capture plant leaf images through their browser without manually uploading a file. This would improve usability and make the system more interactive.

It is recommended to include **disease treatment suggestions** alongside predictions. Providing actionable recommendations such as pesticide usage guidelines or preventive measures will make the system significantly more useful for farmers.

Another improvement is the integration of **IoT-based smart agriculture systems**. Environmental sensor data can be combined with image-based predictions to provide more comprehensive insights into plant health conditions.

The system can also benefit from using advanced deep learning architectures such as **EfficientNet** or **ResNet** to further improve accuracy. However, care must be taken to balance accuracy with server-side computational efficiency for web deployment.

Additionally, implementing **multi-language support** in the web interface can make the system accessible to a wider range of users, especially in rural areas where regional languages are preferred.

It is also recommended to conduct **field testing and validation** under real agricultural conditions. This will help identify practical challenges and improve overall system reliability.

Finally, deploying the system on **scalable cloud platforms** (such as AWS, Google Cloud, or Azure) can enable high availability and concurrent access, allowing multiple users across different regions to use the system simultaneously.

8.3 Conclusion

The Plant Disease Detection system developed in this project successfully demonstrates the application of artificial intelligence in agriculture. By leveraging Convolutional Neural Networks (CNNs), the system is capable of accurately identifying plant diseases from leaf images and delivering real-time predictions through a browser-accessible web interface.

The project addresses the limitations of traditional manual disease detection methods by offering an automated, fast, and efficient solution. The integration of deep learning with web application technology ensures that the system is not only accurate but also accessible and user-friendly — operable from any device with a web browser, without requiring any software installation.

The implementation process covered all essential stages, including dataset preparation, preprocessing, model development, training, evaluation, and deployment. The trained CNN model is served through a lightweight Flask REST API, which processes image uploads from the web frontend and returns predictions in under 1–2 seconds per image.

The results obtained from the project indicate that deep learning models can significantly improve the accuracy and efficiency of plant disease detection. The system provides a scalable solution that can be extended to support additional crops and diseases in the future.

Despite its success, the system has certain limitations, such as dependency on dataset quality and sensitivity to image variations. However, these limitations can be addressed through further research and improvements, as outlined in the recommendations.

In conclusion, this project highlights the potential of artificial intelligence in transforming agricultural practices. By providing an intelligent and accessible web-based tool for plant disease detection, the system contributes to improving crop health, reducing agricultural losses, and supporting sustainable farming. The project serves as a strong foundation for future advancements in precision agriculture and AI-driven farming solutions.

CHAPTER 9
REFERENCES

[1] S. Ramesh, N. M., P. R., P. B. N., S. N., R. Hebbar, and P. V. Vinod,
"Plant Disease Detection Using Machine Learning," *Proceedings of the International Conference on Design Innovations for 3Cs Compute, Communicate, Control (ICDI3C)*, IEEE, 2018.

[2] S. P. Mohanty, D. P. Hughes, and M. Salathé,
"Using Deep Learning for Image-Based Plant Disease Detection," *Frontiers in Plant Science*, vol. 7, Article 1419, 2016, doi:10.3389/fpls.2016.01419.

[3] E. C. Too, L. Yujian, S. Njukia, and L. Yingchun,
"A Comparative Study of Fine-Tuning Deep Learning Models for Plant Disease Identification," *Computers and Electronics in Agriculture*, vol. 161, pp. 272–279, 2019.

[4] X. Sun, G. Li, P. Qu, X. Xie, X. Pan, and W. Zhang,
"Research on Plant Disease Identification Based on CNN," *Cognitive Robotics*, vol. 2, pp. 155–163, 2022.

[5] N. Nigar, H. M. Faisal, M. Umer, O. Oki, and J. M. Lukose,
"Improving Plant Disease Classification With Deep-Learning-Based Prediction Model Using Explainable Artificial Intelligence," *IEEE Access*, vol. 12, 2024, doi:10.1109/ACCESS.2024.3428553.

[6] D. Tejaswi, T. S. Vaishnavi, B. Nandini, P. N. Raju, and B. Deepika,
"Plant Disease Detection Using Deep Learning," *International Journal of Science and Research Archive*, vol. 12, no. 1, pp. 2476–2488, 2024.

[7] W. B. Demilie,
"Plant Disease Detection and Classification Techniques: A Comparative Study of the Performances," *Journal of Big Data*, vol. 11, Article 5, 2024, doi:10.1186/s40537-023-00863-9.

[8] Q. Zou, K. Qu, Y. Luo, D. Yin, Y. Ju, and H. Tang,
"Predicting Diabetes Mellitus With Machine Learning Techniques," *Frontiers in Genetics*, 2018.

[9] M. K. Hasan, M. A. Alam, D. Das, E. Hossain, and M. Hasan,
"Diabetes Prediction Using Ensembling of Different Machine Learning Classifiers," *IEEE Access*, 2020.

[10] J. Soni, U. Ansari, D. Sharma, and S. Soni,
"Predictive Data Mining for Medical Diagnosis," *International Journal of Computer Applications*, vol. 17, no. 8, 2011.

APPENDIX A

Additional Information

Additional screenshots, trained model files, source code samples, and system documentation are available in the project repository. The entire codebase is organized using a modular structure, including separate components for data preprocessing, model training, prediction, and Web application integration.

All modules are properly documented with inline comments to ensure readability and maintainability. The project structure follows standard development practices, making it easy to extend and deploy in real-world applications.